AI System Design & LLMOps

Interview Preparation Guide

100+ Questions & Expert Answers

Dhiraj Patra | Senior Engineering Lead & AI/ML Architect

linkedin.com/in/dhirajpatra | dhirajpatra.github.io

AI System Design

Q1: Design an AI-powered customer support chatbot.

Answer:

A production-grade AI customer support chatbot involves multiple layers: NLU, dialogue management, knowledge retrieval, and response generation.

Core Architecture:

- Frontend: Web/mobile widget → API Gateway → Load Balancer
- Orchestration Layer: Intent classifier + Dialogue state manager
- RAG Pipeline: Vector DB (Pinecone/Weaviate) + LLM (GPT-4/Claude) for answer generation
- Escalation Engine: Rules + ML-based human handoff detection
- CRM Integration: Salesforce/Zendesk for ticket creation and context fetch

Key Design Decisions:

- **Latency:** Use streaming responses, cache frequent Q&A pairs in Redis.
- **Accuracy:** RAG over company knowledge base; fallback to LLM general knowledge.
- **Safety:** Guardrails for PII masking, profanity filter, hallucination detection.
- **Escalation:** Detect frustration signals (repetition, negative sentiment) → human agent.
- **Multi-turn memory:** Conversation window passed per turn; summarize after 10 turns.

Monitoring: Track CSAT, resolution rate, escalation rate, average handle time, hallucination rate.

Q2: Design a document Q&A system for enterprise use.

Answer:

Enterprise document Q&A requires secure ingestion, accurate retrieval, and verifiable sourcing.

Pipeline:

- Ingestion: PDF/DOCX/HTML → Document parser → Chunking (512-1024 tokens, 20% overlap)
- Embedding: OpenAI text-embedding-3-large or BGE-M3 → Stored in Pinecone/pgvector
- Retrieval: Hybrid search (BM25 sparse + dense vector) + reranking (Cohere Rerank)
- Generation: LLM with context window injection + citation enforcement in prompt
- Access Control: Row-level security on vector DB; ACL per document at ingestion

Enterprise-specific concerns:

- **Security:** Data never leaves VPC; on-prem model options (LLaMA, Mistral).
- **Auditability:** Every query logged with retrieved chunks and source citations.
- **Freshness:** Incremental re-indexing on document update; version control on embeddings.
- **Multi-tenancy:** Namespace isolation per department/team in vector DB.

Evaluation: Faithfulness score (answer grounded in docs), context recall, answer relevancy using RAGAS and TruLens framework.

Q3: Design a code generation and review system.

Answer:

A production code gen & review system integrates into CI/CD and IDE workflows.

Components:

- Code Generation: LLM (Claude/GPT-4/CodeLlama) with repository context via RAG over codebase
- Context Retrieval: AST parsing + semantic search over functions/classes in repo
- Review Agent: Static analysis (ESLint, SonarQube) + LLM review in parallel
- Security Scan: SAST tools + LLM for logic vulnerability detection
- Test Generation: Automatic unit test scaffolding from function signatures

Workflow:

- Developer submits PR → CI triggers review pipeline
- AST extracts changed functions → vector search for similar code patterns
- LLM generates: security notes, performance suggestions, style fixes
- Results posted as PR comments with line-level annotations

Key considerations:

- **Hallucination risk:** LLM suggestions validated against compiler/linter before surfacing.
- **Latency:** Parallel execution of analysis tasks; async result posting.
- **Cost:** Use smaller model (GPT-3.5/Haiku) for syntax; large model for logic review only.

Q4: Design a content moderation system using AI.

Answer:

Content moderation at scale requires a multi-tier approach balancing speed, accuracy, and cost.

Architecture Tiers:

- Tier 1 – Rule-based (< 1ms): Blocklist, regex patterns, URL reputation check
- Tier 2 – Fast ML model (< 50ms): Fine-tuned BERT/DistilBERT classifier for categories
- Tier 3 – LLM (< 2s): GPT-4/Claude for nuanced, context-dependent content
- Tier 4 – Human review: Escalated low-confidence or high-stakes decisions

Categories to detect: Hate speech, CSAM, violence, spam, misinformation, self-harm content.

Design decisions:

- **Multi-modal:** Separate pipelines for text (NLP), image (ViT/CLIP), video (frame sampling + audio).
- **Appeals:** User appeal flow triggers human review with full context.
- **Bias audits:** Regular evaluation across demographic groups; fairness metrics tracked.
- **Latency SLA:** Tier 1+2 sync; Tier 3 async with content held pending review.

Metrics: False positive rate (FPR), false negative rate (FNR), average review time, escalation rate.

Q5: Design a real-time AI recommendation system.

Answer:

A real-time recommendation system combines collaborative filtering, content signals, and LLM-powered personalization.

Architecture:

- Feature Store: User features (Feast/Tecton), item features, interaction signals (Kafka streams)
- Candidate Generation: ANN index (FAISS/ScaNN) retrieves top-1000 items in < 10ms
- Ranking Model: Two-tower DNN or LightGBM on 100–200 features → top-50 candidates
- LLM Re-ranker (optional): For cold-start or context-rich scenarios (e.g., search query)
- Serving: Redis-backed pre-computed recs + real-time override on new signals

Key design:

- **Cold start:** Content-based fallback using item embeddings from LLM.
- **Diversity:** Maximal marginal relevance (MMR) to avoid filter bubbles.
- **A/B testing:** Holdout groups per user segment; Bayesian bandit for faster convergence.
- **Latency:** P99 < 100ms end-to-end; pre-compute daily batch recs as fallback.

Q6: Design a multi-modal search system (text, image, video).

Answer:

Multi-modal search maps different modalities into a shared embedding space for cross-modal retrieval.

Embedding Layer:

- Text: OpenAI text-embedding-3 or E5-large
- Image: CLIP ViT-L/14 or Gemini multimodal embeddings
- Video: Frame sampling (1 fps) → CLIP embeddings → temporal average pooling
- Audio: Whisper transcription → text embedding

Index & Retrieval:

- Unified vector DB (Weaviate/Qdrant) with modality-specific namespaces
- Cross-modal queries: Text query retrieves images/videos via shared CLIP space
- Hybrid: Dense vector + BM25 sparse (for text) with score fusion

Infrastructure:

- **Ingestion:** Async pipeline — Kafka → embedding service → vector DB.
- **Query routing:** Detect query modality → select embedding model → query vector DB.
- **Latency:** GPU inference for embedding; cache popular query embeddings.
- **Re-ranking:** Cross-encoder or LLM-as-judge for top-20 results.

Q7: Design an AI-powered email assistant.

Answer:

An AI email assistant handles drafting, summarization, priority sorting, and action extraction.

Features & Architecture:

- Triage & Priority: LLM classifies urgency, sender importance, topic category
- Summarization: LLM summarizes thread → key points, action items, deadlines
- Draft Generation: Context-aware drafts using email history + user writing style
- Calendar Integration: Detect meeting requests → propose slots via Calendar API
- Action Extraction: NER + LLM to extract tasks → push to task manager (Jira/Asana)

Privacy & Security:

- **On-device option:** Small LLM (Phi-3/Mistral 7B) for sensitive enterprises.
- **PII masking:** Strip PII before sending to cloud LLM; re-inject in response.
- **OAuth:** Gmail/Outlook OAuth 2.0; minimal scope (read-write mail only).

User experience:

- One-click draft approval with inline editing
- Learning from user edits to personalize future drafts (fine-tuning or RLHF signal)
- Unsubscribe automation for detected promotional threads

Q8: Design a medical diagnosis assistant using AI.

Answer:

Medical AI must be explainable, auditable, and clearly positioned as decision support — not replacement for clinical judgment.

System Architecture:

- Input: Structured EHR data + unstructured notes + imaging (X-ray, MRI, pathology)
- NLP pipeline: Clinical NLP (Spark NLP Healthcare / AWS Comprehend Medical) for entity extraction
- Diagnosis Model: Differential diagnosis generation via LLM (GPT-4 / MedPaLM) + retrieval from medical KB
- Imaging AI: Specialized CNN/ViT models per modality (chest X-ray, ECG analysis)
- Explainability: SHAP/LIME for structured features; attention maps for imaging

Compliance & Safety:

- **HIPAA:** Encrypted data at rest/transit; audit logs; BAA with LLM provider.
- **FDA:** If used clinically, may require 510(k) clearance; position as 'clinical decision support.'
- **Guardrails:** LLM never makes final diagnosis; always outputs confidence + references.
- **Human oversight:** Mandatory physician review before any clinical action.

Evaluation: Sensitivity/specificity per condition; comparison vs. physician baseline; uncertainty calibration. Ragas, TruLens, PromptFoo evaluation framework.

Q9: Design a fraud detection system powered by LLMs.

Answer:

LLM-powered fraud detection augments traditional ML models with reasoning over unstructured signals.

Architecture:

- Real-time scoring: Gradient boosting (XGBoost) on structured features → decision in < 50ms
- Graph analytics: GNN over transaction network to detect fraud rings
- LLM layer: Analyzes transaction narrative, merchant description, account notes for anomalies
- Alert triage: LLM generates human-readable explanation for analyst review
- Feedback loop: Analyst decisions fed back as labels for model retraining

Key design decisions:

- **Latency:** LLM runs async on high-risk transactions flagged by fast model; not in critical path.
- **Explainability:** LLM generates natural language explanation for every flagged transaction.
- **Imbalanced data:** SMOTE, cost-sensitive learning; optimize for recall (minimize false negatives).
- **Adversarial:** Adversarial training; monitor for distribution shift as fraudsters adapt.

Metrics: Precision, recall, F1 on fraud class; $ saved per month; false positive rate (impacts UX).

Q10: Design an AI-powered data extraction pipeline from unstructured documents.

Answer:

Unstructured document extraction converts PDFs, scans, emails, and HTML into structured data at scale.

Pipeline:

- Ingestion: S3/GCS bucket trigger → message queue (SQS/Pub-Sub)

- OCR: Tesseract (open-source) or AWS Textract / Google Document AI for scanned docs
- Layout parsing: LayoutLM or Donut model for form understanding, table extraction
- Entity extraction: LLM with structured output (JSON schema) for named fields
- Validation: Rule-based checks + confidence scoring; low-confidence → human review queue
- Output: Structured JSON → downstream DB, data warehouse, or API

Key considerations:

- **Multi-format:** Handle PDF, DOCX, XLSX, HTML, images — separate parsers per format.
- **Table extraction:** Camelot/Tabula for PDFs; LLM for complex merged-cell tables.
- **Accuracy:** Measure field-level extraction accuracy; target > 95% for automation.
- **Cost:** Use fast OCR + rule extraction first; invoke LLM only for ambiguous cases.

Scale: Parallel processing workers (Celery/Airflow); async architecture for burst traffic.

Q11: Design a personalized learning assistant.

Answer:

A personalized learning assistant adapts content difficulty, pacing, and format to individual learner profiles.

Core Components:

- Learner model: Knowledge graph tracking mastery per concept (spaced repetition via SM-2)
- Content generation: LLM generates explanations, examples, quizzes tailored to level
- Adaptive assessment: IRT (Item Response Theory) model estimates ability from responses
- Socratic tutor: LLM guides through hints rather than direct answers
- Progress analytics: Dashboard showing learning velocity, weak areas, time-to-mastery

Design decisions:

- **Personalization:** Zero-shot personalization via learner profile in LLM system prompt.
- **Multimodal:** Text explanations + auto-generated diagrams + video recommendations.
- **Engagement:** Gamification (streaks, badges); conversational tutoring interface.
- **Safety:** Age-appropriate content filters; parent/teacher oversight dashboard.

Evaluation: Learning gain (pre/post test delta), engagement metrics, retention at 7/30 days.

Q12: Design an AI system for automated code migration.

Answer:

Automated code migration uses LLMs + AST analysis to translate code between languages or framework versions.

Pipeline:

- Parse source code → AST (using language-specific parser: tree-sitter, Roslyn, JavaParser)
- Semantic analysis: Identify patterns, idioms, deprecated APIs, dependencies
- LLM translation: Few-shot examples of source→target patterns + AST context
- Compilation check: Auto-build migrated code; capture errors → re-prompt LLM
- Test generation: Generate/adapt unit tests for migrated code
- Diff review: Generate human-readable migration summary and PR

Key design:

- **Iterative refinement:** LLM + compiler in a loop until build passes or max iterations.

- **Pattern library:** Pre-built migration patterns for common cases (e.g., Python 2→3, jQuery→React).
- **Partial migration:** Migrate file-by-file; maintain interop layer during transition.
- **Accuracy:** Track compilation success rate, test pass rate, human acceptance rate.

Q13: Design an AI-powered legal document review system.

Answer:

Legal AI must handle domain-specific language, maintain confidentiality, and provide verifiable citations.

Architecture:

- Ingestion: PDF/DOCX contracts → OCR → chunking with clause-boundary awareness
- Clause extraction: LLM identifies standard clauses (IP, termination, liability, payment)
- Risk analysis: Per-clause risk scoring vs. company playbook; flag deviations
- Comparison: Diff against standard templates; highlight unusual provisions
- Q&A interface: Lawyer asks questions → RAG over document → cited answers

Critical design:

- **Attorney-client privilege:** Data isolation per matter; no cross-contamination across clients.
- **Hallucination risk:** Every answer must cite source chunk; confidence threshold required.
- **Jurisdiction awareness:** Prompt includes governing law context; model trained on legal data (Harvey, CaseMine).
- **Audit trail:** Log every query, retrieved context, and LLM response for malpractice protection.

Output: Redline markup, risk heatmap per clause, executive summary for business review.

Q14: Design a conversational AI system with memory across sessions.

Answer:

Persistent conversational memory enables context continuity across separate user sessions.

Memory Architecture:

- Short-term (in-session): Full conversation in context window
- Episode memory: After session, LLM summarizes key facts → stored in user profile DB
- Semantic memory: User facts/preferences embedded → vector DB for retrieval
- Procedural memory: Learned user preferences (communication style, topics to avoid)

Implementation:

- At session start: Retrieve top-k relevant memories via semantic search
- Inject into system prompt: 'What I know about you: [memories]'
- Post-session: LLM extracts new facts → update memory store
- Memory decay: TTL on episodic memories; consolidate over time

Design decisions:

- **Privacy:** User can view/delete all memories; GDPR compliance.
- **Relevance:** Memory retrieval by recency × relevance score, not all memories injected.
- **Conflicts:** New facts overwrite contradictory old facts; timestamp-based resolution.
- **Scale:** Memory per user in isolated namespace; async update pipeline.

Architecture & Operational Design

Q15: How do you design for latency vs quality trade-offs in AI systems?

Answer:

Latency and quality exist on a spectrum — the optimal point depends on use case criticality and user tolerance.

Key levers:

- **Model size:** Smaller models (Haiku/GPT-3.5) are faster; use larger models only when quality is critical.
- **Context length:** Shorter prompts reduce TTFT; compress/summarize context.
- **Streaming:** Stream tokens to user — perceived latency drops even if total time is same.
- **Caching:** Semantic cache (GPTCache) for repeated/similar queries.
- **Async processing:** Non-real-time tasks (report generation) run in background.

Design pattern: Tiered routing

- < 200ms SLA → small local model or cached result
- < 2s SLA → mid-size model (Sonnet/GPT-3.5)
- < 10s SLA → large model (GPT-4/Claude Opus) with full reasoning

Measurement: Track P50/P95/P99 latency separately. Quality via BLEU/BERTScore on sample. Run A/B to find acceptable quality floor at lower latency.

Q16: How do you implement caching strategies for LLM applications?

Answer:

Caching is one of the highest-ROI optimizations for LLM cost and latency.

Cache Tiers:

- **Exact match cache:** Hash of prompt → cached response in Redis. Works for templated queries (FAQs).
- **Semantic cache (GPTCache):** Embed query → find similar cached query (cosine similarity > 0.95) → return cached response.
- **Prefix/KV cache:** Provider-side (Anthropic, OpenAI) caches common prompt prefixes (system prompt). Use prompt caching API.
- **Partial result cache:** Cache retrieved RAG chunks, embeddings, reranked results separately.

Implementation:

- Redis for exact match; Qdrant/Redis with vector support for semantic cache
- TTL based on data freshness requirements (1hr for news queries, 24hr for product FAQs)
- Invalidation: On knowledge base update, flush affected cache entries
- Warm-up: Pre-compute embeddings for top-500 predicted queries at deployment

Cache hit rate of 30–50% is achievable in customer support use cases, cutting costs proportionally.

Q17: How do you design rate limiting and cost management for AI APIs?

Answer:

Without rate limiting and cost guardrails, a single runaway process can exhaust budget instantly.

Rate Limiting:

- Token bucket / sliding window algorithm per user, team, and application
- Separate limits: requests/min, tokens/min, tokens/day
- Priority queues: Premium users served first; batch jobs throttled
- Backpressure: Return 429 with Retry-After; clients implement exponential backoff

Cost Management:

- **Budgets:** Per-user, per-project token budgets with alerting at 80% / hard stop at 100%.
- **Model routing:** Route simple queries to cheap models; complex to expensive ones.
- **Token optimization:** Trim context aggressively; use structured output to reduce verbose responses.
- **Monitoring:** Real-time cost dashboard (token usage × price per token) per API key, endpoint, user.
- **Anomaly detection:** Alert if usage spikes 3x above baseline within 5 minutes.

Governance: FinOps review of AI spend monthly; chargeback to cost centers per team.

Q18: How do you handle failover and fallback strategies for AI systems?

Answer:

AI systems must degrade gracefully rather than fail completely when primary providers are unavailable.

Failover Strategies:

- **Multi-provider:** Primary (OpenAI GPT-4) → Secondary (Anthropic Claude) → Tertiary (Azure OpenAI). LiteLLM proxy handles routing.
- **Model degradation:** GPT-4 unavailable → GPT-3.5-turbo; slightly lower quality but functional.
- **Cached fallback:** Serve stale cached responses for read queries when live model is down.
- **Heuristic fallback:** Rule-based response for predictable queries (FAQ, simple classification).
- **Circuit breaker:** After N consecutive failures, open circuit; route to fallback; retry primary after T seconds.

Implementation:

- Health check loop: Ping provider endpoints every 30s; update routing table
- LiteLLM / Kong AI Gateway for transparent provider failover
- Client retries: 3 retries with exponential backoff (1s, 2s, 4s) before failover
- User communication: Show 'enhanced mode unavailable, using standard responses' rather than error

---

Q19: How do you design an AI system for high availability and fault tolerance?

Answer:

High availability for AI systems means eliminating single points of failure across the entire inference stack.

Infrastructure:

- Multi-AZ deployment of inference servers; load balancer distributes traffic
- Model replicas: Minimum 3 replicas per model; auto-scaling on token throughput
- GPU pool: Preemptible/spot instances for batch; on-demand for real-time
- API Gateway: Kong/AWS API Gateway with health-aware routing

Data Layer:

- Vector DB: Weaviate/Pinecone with replication factor ≥ 3
- Cache: Redis Cluster with sentinel; no single Redis node
- LLM state: Stateless inference (no session state on GPU worker)

Patterns:

- **Bulkhead:** Isolate real-time inference from batch inference; one can't starve the other.
- **Retry with idempotency:** Token IDs ensure same request doesn't generate twice.
- **SLA targets:** 99.9% availability = 8.7hr downtime/year; 99.99% = 52min/year.

Chaos engineering: Quarterly game days to test failover; inject failures in staging.

Q20: How do you design an AI system that gracefully degrades when the model is unavailable?

Answer:

Graceful degradation means the system remains useful at reduced capability rather than failing entirely.

Degradation Tiers:

- Tier 1 (Full): Live LLM inference with RAG — full capability
- Tier 2 (Cached): Serve semantically matched cached responses — 70% of queries handled
- Tier 3 (Heuristic): Rule-based responses for common intents — basic functionality
- Tier 4 (Static): Hardcoded FAQ responses + 'service degraded' notice to users
- Tier 5 (Human): Route to human agent queue — zero AI, full service

Implementation:

- **Feature flags:** Toggle tiers via LaunchDarkly/Flagsmith without deployment.
- **SLO-based switching:** Automatically drop to lower tier when error rate > 5% or p99 > 10s.
- **Transparency:** Inform users when degraded mode is active; set expectations.
- **Recovery:** Canary traffic to primary model before full restoration.

Q21: What are the key considerations for multi-region deployment of AI systems?

Answer:

Multi-region AI deployment addresses data residency, latency, and disaster recovery requirements.

Key Considerations:

- **Data residency:** EU data must stay in EU region (GDPR); financial data per local regulations.
- **Model hosting:** Deploy model replicas in each region; avoid cross-region inference latency.
- **Vector DB sync:** Active-active replication (Weaviate federation) or primary-replica with read routing.
- **Prompt/config sync:** GitOps for prompt versions across regions; deploy simultaneously.
- **Latency:** Route users to nearest region via GeoDNS / Cloudflare; < 100ms RTT target.

Operational challenges:

- Model version consistency across regions — use immutable model registries
- Fine-tuned model weights replicated via CDN or artifact registry
- Monitoring: Centralized observability (Datadog/Grafana) aggregating all regional metrics
- Cost: Cross-region egress fees; route queries within region when possible

---

Q22: Design an AI-powered search engine for an e-commerce platform.

Answer:

E-commerce AI search must balance relevance, personalization, and business rules (promotions, inventory).

Architecture:

- Query understanding: LLM intent classifier (navigational / transactional / informational) + query expansion

- Retrieval: Elasticsearch (BM25 + semantic KNN) + product catalog vector index
- Re-ranking: Personalized ranking model using user history, click-through, purchase signals
- Business rules layer: Boost in-stock, on-sale items; demote out-of-stock after ranking
- Conversational search: Multi-turn refinement ('show me something cheaper in blue')

Key decisions:

- **Cold start:** Content-based ranking on product description embeddings for new users.
- **Real-time signals:** Inventory, price, promotions updated in real-time via Kafka → search index.
- **A/B testing:** Test ranking models; measure CTR, add-to-cart, conversion rate.
- **Spell correction:** SymSpell + query suggestion via LLM for zero-result queries.

Q23: Design an AI gateway/proxy for managing LLM access across an organization.

Answer:

An AI gateway centralizes routing, auth, rate limiting, observability, and cost management for all LLM usage.

Core Features:

- Authentication: API key per team/service; scoped to allowed models and budgets
- Model routing: Route by cost, latency, capability; failover logic built-in
- Rate limiting: Per-key token limits; organization-wide budget enforcement
- Observability: Log every request/response (sanitized); latency, cost, token usage per caller
- PII scrubbing: Strip PII before forwarding; re-inject in response
- Prompt injection defense: Detect and block prompt injection patterns

Implementation stack:

- **Open-source:** LiteLLM proxy, Portkey, or Kong AI Gateway as foundation.
- **Custom extensions:** Org-specific auth, cost allocation, content policies as middleware.
- **Admin UI:** Dashboard for usage by team, model, endpoint; budget management.

Governance: All LLM access through gateway — no direct API calls. Enforced via network policy.

Q24: How do you design a RAG system that handles conflicting information across sources?

Answer:

Conflicting information in RAG is common when sources have different dates, perspectives, or accuracy levels.

Detection:

- Embed retrieved chunks → compare pairwise cosine similarity; low similarity on same topic signals conflict
- LLM in verification pass: 'Do any of these sources contradict each other?'
- Structured conflict detection: Extract claims per chunk → compare entity-attribute pairs

Resolution strategies:

- **Source authority:** Rank sources by credibility (official docs > blog > forum); prefer authoritative.
- **Recency:** Prefer more recent documents when conflict is due to temporal change.
- **Transparency:** Surface conflict to user: 'Source A says X, Source B says Y; here is our assessment.'
- **Abstention:** If conflict unresolvable, decline to answer and surface all conflicting claims.
- **Voting:** If 3+ sources agree vs 1 outlier, weight majority view (with caveat).

Evaluation: Track conflict rate per topic domain; measure user satisfaction when conflict is surfaced vs. hidden.

Q25: How do you approach capacity planning for an AI system?

Answer:

Capacity planning for AI systems requires forecasting token throughput, GPU utilization, and cost under load.

Steps:

- Baseline measurement: Tokens/request (avg prompt + completion), requests/sec, p99 latency per model
- Traffic forecast: Growth rate + seasonal peaks (e.g., 3x on campaign launch days)
- GPU sizing: Tokens/sec per GPU (benchmark per model size) → GPU count for target load
- Memory: KV cache memory = 2 × num_layers × hidden_dim × seq_len × batch_size × dtype_bytes
- Cost model: (GPU cost/hr × GPU count) + API cost if using managed providers

Rules of thumb:

- **70% utilization ceiling:** Plan for headroom; AI inference is spiky.
- **Auto-scaling:** Scale on token throughput metric, not CPU (CPU is idle during GPU inference).
- **Load testing:** Locust/k6 with realistic prompt distributions before production launch.
- **Provider quotas:** Request TPM/RPM quota increases from OpenAI/Anthropic 4+ weeks ahead of launch.

Q26: Design a multi-tenant AI chatbot platform where each business gets a custom chatbot.

Answer:

A multi-tenant chatbot platform must isolate data, allow customization, and scale per-tenant without interference.

Architecture:

- Tenant onboarding: Upload knowledge base → ingest pipeline → isolated namespace in vector DB
- Configuration: Per-tenant: system prompt, persona, allowed topics, escalation rules, branding
- Routing: API key → tenant lookup → inject tenant config + retrieve from tenant's vector namespace
- Isolation: Strict namespace separation; query never crosses tenant boundaries
- Custom models: Tenants can BYOM (bring your own model) via model registry

Operations:

- **Noisy neighbor:** Per-tenant rate limits to prevent one tenant from hogging GPU capacity.
- **Billing:** Token usage metered per tenant; usage-based pricing model.
- **SLA tiers:** Standard (shared GPU) vs. Premium (dedicated GPU) for enterprise tenants.
- **Monitoring:** Per-tenant dashboards for usage, CSAT, escalation rate.

Compliance: SOC2 Type II; per-tenant data deletion on offboarding; GDPR right-to-erasure pipeline.

Q27: Design an AI meeting summarizer system for thousands of meetings daily.

Answer:

At scale, meeting summarization requires async processing, efficient transcription, and structured output.

Pipeline:

- Audio ingestion: Meeting recording → S3 → SQS queue trigger
- Transcription: Whisper large-v3 (self-hosted) or AssemblyAI API; speaker diarization
- Chunking: Transcript split into 10-min segments for long meetings
- Summarization: LLM generates: summary, action items, decisions, follow-ups per segment → merge
- Output: Structured JSON → email to participants + saved in team wiki (Confluence/Notion)

Scale (10,000 meetings/day):

- **Throughput:** Async worker pool (Celery + Redis); 100+ parallel transcription workers.
- **Cost:** Whisper self-hosted on A10G GPU: ~$0.006/min vs. $0.009/min API; ROI at scale.
- **Latency SLA:** Summary delivered within 15 min of meeting end (async acceptable).
- **Storage:** Transcripts in S3 (cold storage after 90 days); summaries in searchable DB.

Q28: Design an AI notification system that prioritizes instead of broadcasting.

Answer:

Intelligent notification systems reduce alert fatigue by personalizing relevance and timing.

Architecture:

- Event stream: All events → Kafka topic
- Relevance scoring: LLM/ML model scores each event per user: P(user cares about this event)
- User profile: Historical engagement, role, stated preferences, active projects
- Batching engine: Group low-priority notifications into digest; send high-priority immediately
- Channel routing: Push (urgent), email digest (daily), Slack (team-relevant), silent (low priority)

Personalization:

- **Cold start:** Role-based defaults; rapid learning from initial engagement.
- **Quiet hours:** User-defined do-not-disturb windows; hold non-urgent until morning.
- **Feedback loop:** Track open/dismiss/snooze signals → update relevance model per user.
- **Context:** Suppress meeting notifications when user's calendar shows 'in meeting'.

Q29: Design an AI-powered anomaly detection system for cloud infrastructure.

Answer:

Cloud infrastructure anomaly detection must handle high-frequency time series data with low false positive rate.

Architecture:

- Data ingestion: Prometheus/CloudWatch metrics → Kafka → feature engineering service
- Models: Prophet for seasonal baseline; LSTM Autoencoder for multivariate anomalies; Isolation Forest for point anomalies
- LLM layer: On anomaly alert, LLM analyzes correlated metrics + logs → generates root cause hypothesis
- Alert correlation: Topology-aware grouping (same service = one alert, not N); deduplication window
- Runbook automation: LLM suggests remediation steps from historical incident database

Key design:

- **False positive control:** Dynamic thresholds per metric; calibrate at 95th percentile of normal.
- **Seasonality:** Day-of-week and time-of-day normalization; holiday calendars.
- **Latency:** < 60s from metric emission to alert; streaming inference on Flink.
- **Feedback:** Engineers mark false positives → model recalibration pipeline weekly.

Q30: Design an AI-powered document processing pipeline for financial institutions.

Answer:

Financial document processing (KYC, loan applications, trade confirmations) requires accuracy, auditability, and regulatory compliance.

Pipeline:

- Ingestion: SFTP/API → encrypted S3 → document classifier (bank statement, ID, invoice, contract)
- Extraction: OCR (AWS Textract) → LayoutLM for form fields → LLM for complex narrative sections
- Validation: Cross-field checks (dates consistent, math correct, signature present)
- Enrichment: Entity resolution against internal CRM; fraud flag lookup
- Human-in-loop: Confidence < 0.90 → route to human review queue with LLM pre-annotation
- Audit trail: Every extraction step logged with model version, timestamp, confidence

Compliance:

- **PCI-DSS:** PAN/CVV masked immediately on ingestion; never stored in plain text.
- **SOX:** Immutable audit log; 7-year retention.
- **Model risk:** SR 11-7 compliance; model validation by independent team; annual review.

Q31: Design an AI dynamic pricing engine.

Answer:

Dynamic pricing optimizes revenue by adjusting prices in real-time based on demand, competition, and inventory signals.

Architecture:

- Signal ingestion: Demand signals (page views, cart adds), competitor prices (web scraping), inventory levels, weather, events
- Demand forecasting: LSTM or Prophet → demand elasticity per SKU per time window
- Pricing model: Reinforcement learning (contextual bandit) optimizing for revenue × conversion
- LLM layer: Explains pricing decisions to business stakeholders in natural language
- Guardrails: Max/min price bounds, margin floors, competitor parity rules — hard constraints

Operations:

- **Update frequency:** High-velocity items (flights, hotels): per-minute; e-commerce: hourly.
- **A/B testing:** Test pricing strategies on geographic cohorts; holdout for causal measurement.
- **Regulatory:** Price gouging detection; display rules (EU Omnibus Directive).
- **Explainability:** Every price change logged with driver signals for merchant transparency.

Q32: Design an AI resume screening system that handles 100K applications per week.

Answer:

At 100K applications/week, the system must be fast, fair, and auditable.

Pipeline:

- Ingestion: PDF/DOCX upload → parse (pyresparser / custom) → normalize to structured schema
- Embedding: Candidate profile → vector embedding; JD → vector embedding
- Initial ranking: Cosine similarity + keyword match → top 20% pass through
- LLM scoring: Evaluate fit across dimensions: skills match, experience relevance, culture signals
- Bias mitigation: Remove name, gender, age, photo before LLM evaluation; fairness audit per demographic
- Recruiter UI: Ranked shortlist with LLM-generated reasoning per candidate

Scale:

- **Throughput:** 100K/week = ~600/hr; async processing with 50-worker pool easily handles this.

- **Bias:** Regular disparate impact analysis (4/5ths rule); human audit on rejected pool.
- **Compliance:** NYC Local Law 144 (AI bias audit for hiring); EEOC guidelines.
- **Appeals:** Rejected candidates can request human review; automated decision must be explainable.

Q33: Design an AI voice assistant architecture.

Answer:

A production voice assistant pipeline converts speech to intent, processes it, and responds in natural voice.

Pipeline:

- Wake word detection: On-device (Picovoice Porcupine) — no cloud call until activated
- Speech-to-Text (STT): Whisper (self-hosted) or Google STT; streaming for low latency
- NLU: Intent classification + entity extraction (Rasa NLU or LLM)
- Dialogue management: State machine for multi-turn; LLM for open-ended conversation
- Action execution: Calendar, music, smart home via tool calling
- Text-to-Speech (TTS): ElevenLabs / Azure Neural TTS for natural voice output

Latency optimization:

- **Streaming STT:** Begin NLU before STT finishes; overlap pipeline stages.
- **TTS streaming:** Begin audio playback as first sentence is generated, not full response.
- **Edge processing:** STT + wake word on device (Raspberry Pi/Pixel); NLU in cloud.
- **Total latency target:** < 1.5s from end of speech to start of spoken response.

Q34: Design a multi-agent workflow system where agents collaborate on complex tasks.

Answer:

Multi-agent systems decompose complex tasks across specialized agents with orchestrated communication.

Architecture:

- Orchestrator agent: Decomposes task into subtasks; assigns to specialized agents; aggregates results
- Specialized agents: Researcher, Coder, Reviewer, Planner — each with specific tools and system prompts
- Communication: Message passing via shared state (Redis/DB) or direct API calls between agents
- Tool layer: Web search, code execution sandbox, DB query, file read/write
- Memory: Shared working memory (task context) + per-agent scratchpad

Frameworks: LangGraph, AutoGen, CrewAI, custom orchestration.

Key design:

- **Coordination:** Orchestrator uses DAG of subtasks; parallel execution where dependencies allow.
- **Loops & termination:** Max iteration limit; convergence check prevents infinite loops.
- **Human-in-loop:** Pause at checkpoints for human approval before irreversible actions.
- **Observability:** Trace every agent action, tool call, message; visualize agent DAG.

Q35: Design a real-time AI transcription system for concurrent audio streams.

Answer:

Real-time transcription at scale (e.g., 10,000 concurrent call center streams) demands GPU efficiency and low latency.

Architecture:

- Audio ingestion: WebRTC/SIP → media gateway → chunked audio (200ms frames) per session
- STT workers: Whisper or wav2vec2 on GPU; each worker handles 20–50 concurrent streams
- Diarization: PyAnnotate audio → speaker labels per segment
- Post-processing: Punctuation restoration, PII redaction, custom vocabulary (domain terms)
- Output: Transcript stream via WebSocket to client; persisted to S3 + DB async

Scale:

- **Auto-scaling:** Scale STT worker pool on stream count metric; 10s scale-up lag acceptable.
- **GPU efficiency:** Batch audio chunks across sessions on same GPU; dynamic batching.
- **Latency target:** < 500ms word-error lag (words appear as spoken with < 500ms delay).
- **Accuracy:** Fine-tune Whisper on domain vocabulary (medical, legal, financial) for < 5% WER.

Q36: Design an AI-powered live streaming content moderation system.

Answer:

Live stream moderation must operate in real-time with < 5s latency to be effective.

Architecture:

- Video: Frame sampling (2 fps) → image classifier (NSFW, violence, hate symbols) → real-time scoring
- Audio: Live ASR (Whisper streaming) → text moderation pipeline in parallel
- Context: Sliding window (last 30s) for context-aware decisions (e.g., medical vs. NSFW)
- Action: Score > threshold → auto-mute audio / blur video → alert human moderator
- Human review: Streamer context card + live feed view for 10-second confirmation

Design:

- **Latency:** Frame inference < 100ms; decision < 500ms; action < 1s from violation.
- **False positive:** Conservative auto-action (mute, not ban); human confirms before permanent action.
- **Appeals:** Streamer can appeal auto-action; human reviews 30s clip around violation.
- **Adversarial:** Detect attempts to fool model (filters, overlays, coded language).

LLMOps & Production Operations

Q37: Explain the AI product lifecycle from ideation to production.

Answer:

The AI product lifecycle has distinct phases that differ significantly from traditional software.

- Phase 1 – Problem definition: Define the AI task, success metrics, and data requirements.
- Phase 2 – Data: Collect, label, and evaluate training/evaluation datasets. Define golden dataset.
- Phase 3 – Prototyping: Prompt engineering / fine-tuning experiments; baseline evaluation.
- Phase 4 – Evaluation: Offline evaluation (benchmarks, human eval); DeepEval, red teaming.
- Phase 5 – Staging: Shadow mode deployment; A/B testing vs. control; load testing.
- Phase 6 – Production: Canary rollout; full release with monitoring and guardrails.
- Phase 7 – Iteration: Continuous monitoring; prompt/model updates; regression testing.

Key difference from traditional ML: Prompts are code — version-controlled, tested, and deployed through CI/CD. The model is a dependency (like a library version), not owned code.

Q38: What is LLMOps, and how does it differ from traditional MLOps?

Answer:

LLMOps is the operational discipline for deploying and managing large language models in production.

Key differences from MLOps:

- **Prompt as artifact:** Prompts are versioned, tested, and deployed like code — no equivalent in traditional MLOps.
- **Evaluation:** Traditional ML uses metrics like AUC/RMSE. LLMs need semantic similarity, human eval, LLM-as-judge, DeepEval, Promptfoo.
- **No training loop (usually):** Most LLMOps uses pre-trained models; operational focus is on prompts and RAG, not gradient descent.
- **Hallucination:** No equivalent problem in traditional ML classifiers; requires specific detection and guardrail systems.
- **Cost model:** Pay-per-token rather than compute cost — cost management built around token economics.
- **Non-determinism:** Same input → different output; evaluation must handle this.

Shared with MLOps: Data versioning, monitoring, drift detection (distribution shift in inputs), CI/CD pipelines, model registry.

Q39: How do you serve LLMs in production?

Answer:

LLM serving in production requires optimizing for throughput, latency, and cost simultaneously.

Serving Frameworks:

- **vLLM:** PagedAttention for 3–5× throughput improvement; continuous batching; OpenAI-compatible API.
- **TGI (Text Generation Inference):** HuggingFace's optimized server; FlashAttention-2; tensor parallelism.
- **TensorRT-LLM:** NVIDIA's framework; maximum performance on A100/H100.
- **Managed:** OpenAI API, Anthropic API, Bedrock, Vertex AI — zero ops overhead.

Infrastructure:

- GPU selection: A100 80GB for 70B models; A10G 24GB for 7B–13B models
- Tensor parallelism: Shard model across multiple GPUs for models > single GPU VRAM
- KV cache management: PagedAttention prevents memory fragmentation; critical for long contexts
- Quantization: INT8/INT4 reduces memory footprint; AWQ/GPTQ for quality-preserving quantization

Observability: GPU utilization, tokens/sec, queue depth, cache hit rate, P99 latency.

Q40: What is model quantization?

Answer:

Quantization reduces model precision (e.g., FP32 → INT8 → INT4) to decrease memory footprint and increase inference speed.

Types:

- **Post-Training Quantization (PTQ):** Quantize after training; fast but may reduce quality.

- **Quantization-Aware Training (QAT):** Train with quantization simulation; better accuracy preservation.
- **AWQ (Activation-aware Weight Quantization):** Protects salient weights; best quality at INT4.
- **GPTQ:** Layer-wise quantization with second-order information; standard for open-source LLMs.
- **GGUF/GGML:** llama.cpp format; CPU-friendly; widely used for local deployment.

Tradeoffs:

- INT8: ~0.5% quality loss; 2× memory reduction; 1.5–2× speed improvement
- INT4: ~1–3% quality loss; 4× memory reduction; 3–4× speed improvement
- FP16: Standard baseline; half memory of FP32; negligible quality loss

Rule of thumb: Use INT8 for production inference; INT4 for edge/consumer GPU deployment.

Q41: How do you monitor LLM applications in production?

Answer:

LLM monitoring tracks both infrastructure metrics and model quality metrics continuously.

Infrastructure Metrics:

- Latency: TTFT (time to first token), TPOT (time per output token), total request time, P50/P95/P99
- Throughput: Tokens/second, requests/minute, queue depth
- Cost: Token usage (input + output) × price; cost per user, per feature, per day
- Error rates: 4xx, 5xx, timeout rates; provider error breakdown

Quality Metrics:

- Hallucination rate: Automated detection via LLM judge or NLI model
- Response relevance: Semantic similarity of response to query
- User feedback: Thumbs up/down, CSAT scores, escalation triggers
- Input drift: Distribution shift in query topics, length, language over time

Tools: LangSmith, Arize AI, Weights & Biases, Datadog LLM Observability, custom dashboards.

Alerting: PagerDuty alerts for error rate > 1%, latency P99 > 5s, cost spike > 2× baseline.

Q42: What is LLM observability?

Answer:

LLM observability extends traditional observability (metrics, logs, traces) with LLM-specific signals to understand model behavior.

Three pillars:

- **Traces:** End-to-end request trace — prompt → retrieval → LLM call → post-processing → response. Each step timed and logged.
- **Metrics:** Token counts, latency distributions, cost per call, hallucination score, feedback ratings.
- **Logs:** Full prompt + completion stored (with PII masking); tagged by user, session, feature.

LLM-specific additions:

- Prompt lineage: Which prompt version produced which output
- RAG tracing: Which chunks were retrieved, ranked, and injected
- Model version: Exact model version (including API provider version) per request
- Evaluation scores: Automated quality scores attached to each request. DeepEval, Promptfoo.

Tools: LangSmith (LangChain), Helicone, Langfuse (open-source), Arize Phoenix, W&B Traces.

Goal: Debug 'why did this response fail?' in < 5 minutes using traces.

Q43: What are guardrails for LLMs, and how do you implement them?

Answer:

Guardrails are safety controls that prevent LLMs from producing harmful, inaccurate, or policy-violating outputs.

Types:

- **Input guardrails:** Block prompt injection, PII in prompt, off-topic queries, jailbreak attempts.
- **Output guardrails:** Block profanity, harmful content, PII leakage, hallucinated citations.
- **Behavioral guardrails:** Enforce response format (JSON schema), length limits, citation requirements.

Implementation:

- Rule-based: Regex, keyword blocklists, input length limits — fast, zero cost
- Classifier-based: DistilBERT toxic classifier, OpenAI Moderation API — balance of speed and accuracy
- LLM-based: Secondary LLM evaluates primary LLM output — high accuracy, higher latency and cost
- Constitutional AI: System prompt includes rules the model self-checks against

Frameworks: NeMo Guardrails (NVIDIA), Guardrails AI, LlamaGuard (Meta), Presidio (PII).

Pattern: Input guardrail → LLM → Output guardrail. Reject at either stage; log all rejects for analysis.

Q44: How do you implement content filtering for AI outputs?

Answer:

Content filtering is a post-generation check that blocks harmful, off-policy, or inaccurate content before it reaches the user.

Layer 1 – Fast classifiers (< 10ms):

- Profanity/toxicity: perspective-api, Detoxify
- PII detection: Presidio, AWS Comprehend PII
- Off-topic: Embedding similarity to allowed topic space

Layer 2 – LLM judge (< 500ms):

- Secondary LLM evaluates: Is this response safe? Accurate? On-policy?
- Returns: Pass / Fail + reason code

Layer 3 – Human review queue:

- Borderline cases; novel violation patterns for policy team review

Implementation flow:

- Run layers in parallel when possible to minimize added latency
- On fail: Return safe fallback message + log violation with full context
- Feedback loop: Violations reviewed weekly → update classifier training data

Metrics: FPR (false blocks), FNR (missed violations), average filter latency.

Q45: How do you estimate the cost of running an AI-powered feature in production?

Answer:

Cost estimation requires modeling token usage, call patterns, and infrastructure overhead.

Formula: Monthly cost = (daily requests × avg input tokens × input price) + (daily requests × avg output tokens × output price) × 30

Example (GPT-4o at $5/$15 per 1M tokens):

- 10,000 requests/day, 1,000 input tokens, 500 output tokens avg
- Input cost: 10K × 1K × $0.000005 = $50/day
- Output cost: 10K × 500 × $0.000015 = $75/day
- Total: ~$125/day = $3,750/month

Other cost components:

- **Embedding:** Lower cost; estimate separately if using RAG.
- **Infrastructure:** GPU servers for self-hosted models; include amortized cost.
- **Caching savings:** Apply cache hit rate reduction (30–50% typical).
- **Monitoring/tooling:** LangSmith, Datadog add-ons — factor in.

Build a cost model spreadsheet with sliders for traffic growth, cache hit rate, and model selection.

Q46: How do you optimize LLM inference costs in production?

Answer:

Cost optimization for LLM inference has multiple levers across model selection, prompt design, and infrastructure.

Prompt-level:

- Trim context: Remove redundant conversation history; summarize instead of full history
- System prompt caching: Use Anthropic prompt caching for repeated system prompts (90% cache discount)
- Concise instructions: Shorter system prompts = fewer input tokens every call

Model-level:

- Right-size: Use GPT-3.5/Claude Haiku for simple classification; GPT-4 only for complex reasoning
- Semantic routing: Route query type to appropriate model automatically
- Quantized models: Self-host INT4/INT8 quantized model for high-volume, cost-sensitive use cases

Infrastructure-level:

- Semantic caching: 30–50% of similar queries served from cache
- Batch processing: Group non-urgent requests; batch API (50% discount on OpenAI)
- Spot/preemptible GPUs: 60–70% cost reduction for batch workloads

Target: Monthly cost per user < $X defined at product planning; enforce with per-user budgets.

Q47: How do you implement A/B testing for LLM systems?

Answer:

A/B testing for LLMs requires handling non-determinism and selecting appropriate evaluation metrics.

Setup:

- Define variants: Prompt A vs Prompt B, or Model A vs Model B
- Traffic split: Random user-level assignment (not request-level, for consistency)
- Holdout: 10–20% control; 80–90% treatment — or 50/50 for faster results

Evaluation:

- **Online metrics:** User satisfaction (thumbs up/down), task completion rate, session length, escalation rate.
- **Offline metrics:** LLM-as-judge rating on sampled responses; BLEU/BERTScore if applicable.

- **Guardrail metrics:** Ensure safety/policy compliance doesn't degrade in treatment.

Statistical rigor:

- Use t-test or Mann-Whitney U for continuous metrics; chi-squared for binary
- Run until $p < 0.05$ with power ≥ 0.8; avoid peeking early (multiple testing problem)
- Use Bayesian A/B testing for faster decisions with calibrated uncertainty

Tooling: LaunchDarkly for traffic split; Statsig or Optimizely for analysis.

Q48: What is CI/CD for AI applications, and how does it differ from traditional CI/CD?

Answer:

CI/CD for AI applications extends traditional pipelines with evaluation gates, model validation, and prompt testing.

Traditional CI/CD: Code → build → unit test → integration test → deploy.

AI CI/CD adds:

- **Prompt regression tests:** Automated evaluation of prompt changes against golden dataset before merge.
- **Model validation:** On model version update, run full evaluation suite; fail pipeline if quality drops > threshold.
- **Data pipeline tests:** Test embedding pipeline, chunking, retrieval quality for RAG changes.
- **Shadow deployment:** New model/prompt runs alongside production; compare outputs before traffic switch.
- **Evaluation as code:** Eval datasets, metrics, and thresholds versioned in git alongside prompts.

Pipeline stages:

- PR: Run eval suite on changed prompts/code; must pass threshold to merge
- Staging: Full eval + load test; human review of sampled outputs
- Production: Canary (5%) → monitor metrics → full rollout or rollback

Tools: GitHub Actions + LangSmith evals, Weights & Biases, MLflow.

Q49: How do you version and manage prompts in production?

Answer:

Prompt management is analogous to code versioning — prompts are artifacts that must be tracked, tested, and deployed.

Version control:

- Store prompts in git as YAML/JSON files alongside code
- Each prompt has: name, version, template, parameters, model, evaluation criteria
- Semantic versioning: major.minor.patch (major = breaking intent change)

Tooling:

- **Langfuse:** Open-source prompt management with version history and A/B testing.
- **LangChain Hub:** Centralized prompt registry with sharing.
- **Custom registry:** DB table: prompt_id, version, content, created_at, author, status (draft/staging/prod).

Workflow:

- Engineer edits prompt → PR → automated eval run → review → merge to staging → eval → prod deploy
- Production deployment via feature flag (not code deploy) for instant rollback
- Rollback: Flag toggle returns to previous prompt version in < 1 minute

Audit: Every production request logs prompt_version; any output can be traced to exact prompt.

Q50: What is model versioning, and how do you handle model rollbacks?

Answer:

Model versioning tracks which model (and version) produced each output — critical for debugging and compliance.

Versioning strategy:

- Tag every model artifact: model_name/version/quantization (e.g., llama3-70b/v1.2/int8)
- Model registry: MLflow, W&B, or Vertex AI Model Registry — stores weights + metadata
- Immutable versions: Once tagged, model artifact never changes; create new version for changes

Deployment:

- Blue-green: Run old version alongside new; switch traffic when satisfied
- Canary: 5% traffic to new version; monitor quality metrics; ramp up
- Shadow: New model runs in parallel, outputs logged but not served — zero user risk

Rollback:

- Automated: If error rate or quality metric drops below threshold → auto-revert to last good version
- Manual: Traffic routing switch in API gateway — < 5 minutes for full rollback
- Post-mortem: Log which model version caused issue; investigate before re-promoting

Every production response must include model_version in metadata for audit and debugging.

Q51: How do you implement rate limiting and throttling for LLM APIs?

Answer:

Rate limiting protects the system from abuse, cost overruns, and noisy neighbors.

Algorithms:

- **Token bucket:** Bucket fills at R tokens/sec; request consumes N tokens; burst allowed up to bucket capacity.
- **Sliding window:** Count requests in rolling 60-second window; reject if over limit.
- **Fixed window:** Simple counter per minute reset; susceptible to boundary bursts.

Dimensions to rate limit:

- Requests per minute (RPM) — control parallelism
- Tokens per minute (TPM) — control cost and provider API limits
- Tokens per day (TPD) — enforce budget

Implementation:

- Redis INCR with TTL for distributed rate limit counters across pods
- Lua script in Redis for atomic check-and-increment
- Return 429 with Retry-After header; clients implement exponential backoff + jitter

Priority tiers: Premium users get 10× higher limits; background jobs throttled to 10% of real-time.

Q52: How do you handle model updates and migrations without downtime?

Answer:

Zero-downtime model updates require careful traffic management and validation before full rollout.

Strategy:

- **Blue-green deployment:** New model (green) deployed alongside current (blue); traffic switch is atomic; rollback by switching back.
- **Canary release:** 5% → 20% → 50% → 100% traffic over hours/days; monitor at each stage.
- **Shadow mode:** New model receives all traffic but responses not served; validate quality offline.

Compatibility concerns:

- Prompt compatibility: New model may interpret existing prompts differently — test on golden dataset first
- Output schema: If new model changes output format, update parsers before traffic migration
- Context length: If new model has different context limit, adjust chunking/compression accordingly

Process:

- Week 1: Shadow mode + offline evaluation
- Week 2: Canary 5% → 20%; human review of sampled outputs
- Week 3: 50% → 100% with automated quality monitoring
- Rollback trigger: Quality metric drop > 5% or error rate spike → auto-revert

---

Q53: What is the role of feature flags in AI deployments?

Answer:

Feature flags enable safe AI feature releases, fast rollbacks, and controlled experimentation without code deployments.

Use cases in AI:

- **Prompt rollout:** New prompt version behind flag — deploy to 5% users first.
- **Model switch:** Toggle between GPT-4 and Claude without code change or downtime.
- **Feature enable:** New AI feature (e.g., AI-generated summaries) enabled per user segment.
- **Kill switch:** Instantly disable a malfunctioning AI feature across all users.
- **A/B test:** Flag controls treatment assignment for prompt experiments.

Implementation:

- Tools: LaunchDarkly, Flagsmith, Unleash, or simple Redis-backed flag service
- Targeting: Flag can be ON for 'beta users', 'US region', or specific user IDs
- Hierarchy: Organization flag → Team flag → User flag (most specific wins)

AI-specific discipline:

- Every new prompt or model in production is behind a flag initially
- Automated eval must pass before flag can be set to > 50% traffic
- Runbook documents rollback procedure for each AI feature flag

---

Q54: How do you implement logging and tracing for LLM applications?

Answer:

Comprehensive logging and distributed tracing are essential for debugging complex LLM pipelines.

What to log per request:

- Request ID, user ID (hashed), session ID, timestamp
- Prompt version, model ID, model version, temperature, max_tokens
- Input token count, output token count, cost

- Latency (total, TTFT, per pipeline stage)
- Response (with PII masked), quality scores, user feedback
- Retrieved chunks (for RAG), relevance scores

Tracing:

- OpenTelemetry spans for each pipeline stage: retrieval, reranking, LLM call, post-processing
- Parent-child span relationships; trace_id propagated across services
- Visualize in Jaeger or Datadog APM as waterfall diagram

Tools: Langfuse, LangSmith, Helicone, OpenLLMetry (OpenTelemetry for LLMs).

Retention: Full logs 30 days; compressed summary logs 12 months; compliance logs 7 years.

Q55: How do you handle PII and sensitive data in LLM inputs and outputs?

Answer:

PII handling in LLM applications requires detection, masking, and governance at every data boundary.

Input handling:

- Detect PII before sending to LLM: Microsoft Presidio (open-source), AWS Comprehend, Google DLP
- Pseudonymize: Replace 'John Smith, john@acme.com' with 'PERSON_1, EMAIL_1' → re-inject after response
- Contractual: Ensure LLM provider has Data Processing Agreement (DPA); opt out of training data use

Output handling:

- Scan LLM output for PII before returning to user (model may hallucinate real-sounding data)
- Block responses containing detected SSN, credit card, phone number patterns

Storage:

- Never store raw prompts/completions if they contain PII — store pseudonymized versions
- Encryption at rest (AES-256); field-level encryption for PII columns
- Data residency: Keep EU user data in EU region (GDPR Article 46)

On-prem option: For highly sensitive use cases (healthcare, legal), run LLM on-premises — data never leaves.

Audit: Log PII detection events; regular audits of what PII types are encountered in production.

Q56: What is a gateway pattern for LLM API management?

Answer:

The LLM gateway pattern centralizes all LLM API calls through a single proxy that adds cross-cutting concerns.

Gateway responsibilities:

- **Authentication:** Validate caller API key; map to tenant/user/team for cost allocation.
- **Authorization:** Enforce which models and features each caller can access.
- **Rate limiting:** Token and request limits per caller; organization budget enforcement.
- **Routing:** Intelligent routing to providers based on cost, latency, capability, and availability.
- **Observability:** Unified logging, tracing, and cost tracking for all LLM traffic.
- **PII scrubbing:** Strip sensitive data before forwarding to external providers.
- **Caching:** Semantic cache to serve repeated queries without LLM call.
- **Failover:** Auto-switch to backup provider on errors or rate limits.

Options:

- Open-source: LiteLLM proxy (most popular), PortKey, BricksLLM
- Commercial: Kong AI Gateway, Cloudflare AI Gateway, Azure API Management with AI extensions

Principle: No service calls LLM APIs directly; all traffic routes through gateway. Enforced via network policy.

Q57: How do you implement streaming responses for real-time AI applications?

Answer:

Streaming dramatically reduces perceived latency by delivering tokens progressively rather than waiting for the full response.

Server-side:

- LLM API streaming: Set stream=True (OpenAI) / stream=True (Anthropic); receive SSE (Server-Sent Events)
- Backend streams to client: Use FastAPI StreamingResponse or Node.js res.write()
- Format: SSE format — 'data: {"token": "Hello"} '

Client-side:

- EventSource API (browser) or fetch with ReadableStream to consume SSE
- Append tokens to UI as they arrive; show typing indicator between tokens
- Handle [DONE] signal to close stream

Challenges:

- **Guardrails:** Can't run output guardrails until stream completes — buffer and check, or use streaming-compatible classifiers.
- **Error handling:** Stream can fail mid-response; client must handle partial content gracefully.
- **Load balancing:** Long-lived streaming connections; use sticky sessions or connection draining.
- **Cancellation:** Client disconnect → cancel upstream LLM call to avoid wasted tokens.

Perceived latency improvement: From 5s wait to first token in < 500ms — transformative UX improvement.

Q58: What are the key SLAs and metrics for production AI systems (latency, throughput, availability)?

Answer:

Production AI systems require explicit SLAs covering latency, throughput, availability, and quality.

Latency SLAs:

- TTFT (Time to First Token): < 500ms P95 for streaming; < 200ms for cached responses
- Total response time: < 3s P95 for standard; < 10s for complex reasoning
- Embedding generation: < 100ms P95

Throughput:

- Define: Requests per second (RPS) and tokens per second (TPS) the system sustains
- Burst capacity: Handle 3× average load for 5-minute spikes

Availability:

- 99.9% (8.7hr/year downtime) — standard SaaS
- 99.95% — premium enterprise; requires active-active multi-region

Quality SLAs:

- Hallucination rate: < 2% of responses contain factual errors (measured via sampling)
- Policy violation rate: < 0.01% of responses violate content policy

- User satisfaction: CSAT ≥ 4.0/5.0

Error budget: 100% - availability SLA = error budget. Use for planned maintenance and experiments.

Q59: How do you implement fallback strategies when the primary model is unavailable or rate-limited?

Answer:

Fallback strategies must balance quality degradation, cost, and transparency to users.

Fallback chain:

- Level 1: Retry primary model (3× with exponential backoff) — handles transient errors
- Level 2: Route to secondary provider (e.g., Anthropic → OpenAI) — same quality tier
- Level 3: Downgrade model within same provider (GPT-4 → GPT-3.5) — lower quality
- Level 4: Serve cached response for similar query — zero LLM cost, stale data risk
- Level 5: Rule-based/template response — minimal but functional
- Level 6: 'Service temporarily limited' message + queue request for later processing

Implementation:

- Circuit breaker per provider: Trip after 5 consecutive errors; test every 30s
- LiteLLM router handles multi-provider fallback transparently
- Log fallback events with reason for capacity planning

Rate limit fallback specifically:

- Detect 429 response → immediate fallback (no retry wait on current provider)
- Request quota increase proactively when utilization > 70%

Q60: How do you implement structured output from LLMs reliably in production?

Answer:

Structured output ensures LLM responses are machine-parseable, enabling downstream processing without fragile string parsing.

Approaches:

- **Native JSON mode:** OpenAI (response_format: json_object) and Anthropic guarantee valid JSON — use this first.
- **Function calling / tool use:** Define JSON schema as tool input; LLM fills in values. Most reliable for complex schemas.
- **Instructor library:** Python library that wraps OpenAI/Anthropic; uses Pydantic models to validate and retry automatically.
- **Prompt engineering:** Include schema in prompt + few-shot examples; parse with json.loads() + retry on failure.

Production hardening:

- Retry logic: On JSON parse error, re-prompt with error message and ask to correct
- Schema validation: Pydantic/Zod validation after parsing; reject and retry if schema invalid
- Partial recovery: If field missing, use default value rather than failing entire request
- Fallback: After 3 retries, return error to caller or use rule-based default

Log schema validation failures — patterns indicate prompt quality issues.

Q61: How do you handle long contexts efficiently in production (context compression, prefix caching)?

Answer:

Long contexts increase cost linearly and latency quadratically (attention is $O(n^2)$) — compression is essential.

Strategies:

- **Sliding window:** Keep last N tokens; discard older context. Simple but loses early context.
- **Summarization compression:** Periodically summarize old conversation turns; replace N tokens with M (M << N) summary.
- **RAG instead of full context:** Don't inject entire document; retrieve only relevant chunks. Most effective strategy.
- **Selective attention:** Keep: system prompt, recent turns, relevant retrieved context. Drop: verbose mid-conversation filler.
- **Prefix caching:** Anthropic and OpenAI cache repeated system prompts — saves cost and reduces latency on repetitive calls.

Technical:

- Flash Attention 2: Reduces memory from $O(n^2)$ to $O(n)$; enables longer contexts on same GPU
- Sparse attention: Longformer/BigBird patterns for very long documents
- Context budget: Set explicit context budget (e.g., max 8K tokens) and enforce at application layer

Monitor: Track average context length per request; alert if growing over time (memory leak pattern).

Q62: What is semantic routing, and how do you implement it in a multi-model system?

Answer:

Semantic routing directs queries to the most appropriate model or handler based on query meaning, not just keywords.

How it works:

- Embed the incoming query using a lightweight embedding model
- Compare to pre-defined route embeddings (clusters of example queries per route)
- Route to handler with highest cosine similarity above threshold

Implementation:

- **SemanticRouter library:** Python library (Aurelio AI) — define routes with example utterances; routes embedded at startup; fast routing at inference time.
- **Custom:** Maintain route embeddings in memory (FAISS or in-memory list); cosine similarity lookup in < 5ms.

Routing table example:

- Simple factual queries → GPT-3.5-turbo (cheap, fast)
- Code generation → Claude Sonnet (strong coding)
- Complex reasoning → GPT-4 / Claude Opus (highest capability)
- Document Q&A → RAG pipeline
- Off-topic queries → rejection handler

Benefits: 40–60% cost reduction by routing simple queries to cheap models; quality preserved for complex tasks.

Q63: How do you manage secrets and API keys securely in LLM applications?

Answer:

API key leakage is a critical security risk — a leaked LLM key can generate thousands of dollars in charges.
Storage:

- **Never in code:** No hardcoded keys; no keys in git history ever.
- **Secret manager:** AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault — the gold standard.
- **Environment variables:** For local development; injected by deployment system at runtime.
- **k8s secrets:** Encrypted etcd storage; access via pod service account, not hardcoded.

Rotation:

- Rotate API keys every 90 days; automated rotation for provider keys where supported
- On suspected leak: rotate immediately; audit usage logs for unauthorized calls

Access control:

- Principle of least privilege: Each service has its own key with minimal scopes
- Key per environment: Dev/staging/prod use separate keys; never share prod key
- Audit: Cloud provider logs all secret access events; alert on unusual access patterns

Detection: GitGuardian or similar scans all commits for accidental key commits; block merge on detection.

Troubleshooting & Scaling

Q64: Your LLM API has latency spikes during peak hours. How do you stabilize it?

Answer:

Latency spikes during peak hours typically indicate resource contention, provider throttling, or queue buildup.
Diagnosis:

- Check provider status page — are they degraded?
- Plot latency vs. time vs. concurrent requests — does spike correlate with traffic peak?
- Check token queue depth — are requests waiting before even reaching LLM?
- Check TTFT separately — long TTFT = provider latency; long generation = token count issue

Solutions:

- **Pre-scale:** Scale inference capacity 30 min before predicted peak (schedule-based autoscaling).
- **Request queue:** Add async queue (Celery/SQS) with priority; shed low-priority requests at peak.
- **Caching:** Warm semantic cache before peak; reduce LLM calls significantly.
- **Multi-provider:** Load balance across OpenAI + Anthropic + Azure OpenAI at peak.
- **Reduce token count:** Shorten prompts during peak; use smaller context window.
- **Circuit breaker:** If p99 > 10s, activate degraded mode with cached/template responses.

Q65: Your LLM costs are too high in production. How do you reduce costs without degrading quality?

Answer:

Cost reduction is a layered problem — each lever has different impact and implementation effort.
Quick wins (1–2 days):

- Enable semantic caching (GPTCache/custom) — 30–50% reduction for repetitive queries
- Enable prompt prefix caching (Anthropic/OpenAI) — up to 90% discount on cached prefix tokens

- Switch non-critical queries from GPT-4 to GPT-3.5 or Claude Haiku

Medium effort (1–2 weeks):

- Implement semantic router — route query complexity to appropriate model
- Trim system prompts — every 100 tokens saved × N daily calls = significant savings
- Summarize conversation history instead of passing full context

Strategic (1+ month):

- Fine-tune smaller model on your task — match 90% of GPT-4 quality at GPT-3.5 cost
- Self-host quantized model for high-volume, cost-sensitive use cases
- Batch non-real-time jobs through OpenAI Batch API (50% discount)

Measurement: Track cost per successful outcome (not just cost per token) — quality matters.

Q66: Your application is hitting LLM provider rate limits during peak hours. How do you handle it?

Answer:

Rate limit hits (429 errors) require both immediate mitigation and longer-term capacity planning.

Immediate:

- Implement exponential backoff with jitter on 429 responses (1s, 2s, 4s, 8s)
- Add request queue with priority — shed or delay low-priority requests
- Activate secondary provider (Azure OpenAI / Anthropic) as overflow

Short-term:

- Request quota increase from provider (OpenAI Enterprise support — 2–5 day SLA)
- Distribute load across multiple API keys / organizations (check ToS compliance)

Long-term:

- Capacity plan: TPM limit × peak hour fraction = required quota; request 1.5× buffer
- Smooth traffic: Spread batch jobs across off-peak hours; avoid burst patterns
- Multi-provider baseline: Always run with at least 2 providers so failover is instant

Monitoring: Alert when utilization > 70% of quota limit — proactive, not reactive.

Q67: Your application depends on one LLM provider. How do you switch providers without downtime?

Answer:

Provider migration requires abstraction, compatibility testing, and gradual traffic shifting.

Preparation:

- Abstract LLM calls behind an interface (LiteLLM or custom adapter) — no provider-specific code in business logic
- Identify prompt compatibility gaps: New provider may respond differently to existing prompts
- Run both providers in shadow mode: Compare outputs on 1% of production traffic

Migration process:

- Shadow mode (1 week): Validate output quality equivalence on real traffic
- Canary (5%): Send small traffic slice to new provider; monitor quality and errors
- Gradual ramp: 5% → 20% → 50% → 100% over 2–4 weeks
- Full cutover: Disable old provider; keep 30-day rollback window

Gotchas:

- **Token limits:** Different providers have different context windows; update chunking logic.
- **Response format:** Tool calling / function calling format differs across providers.
- **Cost:** Re-validate cost model for new provider pricing.
- **Latency:** Different providers have different regional latency profiles.

Q68: Your AI system handles 100 requests/sec but crashes at 5000. How do you scale for concurrent requests?

Answer:

Scaling from 100 to 5,000 RPS requires horizontal scaling, GPU optimization, and architectural changes.

Bottleneck identification:

- Load test to find bottleneck: Is it GPU (inference), CPU (pre/post-processing), memory, or network?
- Profile at 100 RPS baseline; incrementally increase; find failure point and failure mode

GPU scaling:

- Horizontal: Add GPU nodes behind load balancer; stateless inference servers scale linearly
- Batch inference: Dynamic batching (vLLM continuous batching) increases GPU utilization
- Tensor parallelism: Shard large models across GPUs for memory-bound cases

Architecture:

- Request queue (Redis/SQS): Buffer bursts; decouple ingestion from inference
- Auto-scaling: HPA in Kubernetes on GPU utilization metric (target 70%)
- Pre-compute: For predictable queries, pre-generate responses in batch overnight

Rule of thumb: 5000 RPS at 500 token avg = 2.5M tokens/sec; benchmark your GPU server TPS first to estimate required GPU count.

Q69: A traffic spike brings down your AI system. How do you handle peak traffic?

Answer:

Traffic spike resilience requires proactive capacity planning, load shedding, and queue-based buffering.

Immediate response (minutes):

- Activate load shedding: Reject low-priority requests with 503 + Retry-After
- Activate fallback tier: Serve cached/template responses to reduce LLM load
- Scale out: Trigger manual scale-out of inference fleet

Prevention architecture:

- **Request queue:** SQS/Kafka queue between API gateway and inference workers; absorbs bursts.
- **Autoscaling:** Scale on queue depth metric; 60-second scale-out lag is acceptable with queue buffer.
- **Rate limiting:** Per-user and global limits prevent single event from flooding system.
- **Circuit breaker:** Open on overload; shed traffic intelligently before full crash.
- **Predictive scaling:** Scale up before known spikes (product launch, marketing campaign).

Post-incident: Analyze traffic profile; adjust autoscaling policies; increase baseline capacity if recurring.

Q70: One LLM provider outage took down your entire system. How do you eliminate single points of failure?

Answer:

Single provider dependency is a critical architectural risk — LLM providers have outages like any cloud service.

Multi-provider strategy:

- Always integrate ≥ 2 providers (OpenAI + Anthropic + Azure OpenAI as backup)
- Active-active: Split traffic 70/30 across providers; failover is just adjusting split
- Active-passive: Primary handles all traffic; passive on standby; health check triggers failover

Implementation:

- LiteLLM router: Unified interface across all providers; built-in failover logic
- Health check loop: Test each provider endpoint every 30 seconds; update routing table
- Circuit breaker per provider: Trip after 3 consecutive errors; test every 60s

Additional resilience:

- Cache layer: Semantic cache serves 30–40% of queries without LLM call
- Self-hosted fallback: Local Llama/Mistral model for basic queries when all providers down
- Graceful degradation tiers: Define what features work at each degradation level

Testing: Chaos engineering — inject provider failure quarterly; validate failover works correctly.

Q71: Your multi-LLM pipeline fails when one model in the chain breaks. How do you handle orchestration failure?

Answer:

Multi-model pipelines need fault isolation, retry logic, and fallback paths at each node.

Patterns:

- **Saga pattern:** Each step can compensate (undo) previous steps on failure; ensures consistency.
- **Checkpoint/resume:** Save intermediate results; on failure, resume from last checkpoint not from start.
- **Bulkhead:** Each pipeline stage runs in isolated process pool; one failure doesn't cascade.
- **Timeout per stage:** Each LLM call has an independent timeout; stuck step doesn't freeze pipeline.

Fallback per stage:

- Extraction step fails → use rule-based extractor as fallback
- Summarization step fails → return original text with warning
- Classification step fails → use default/neutral classification

Observability:

- Trace ID propagated through all pipeline stages
- Each stage logs: input, output, duration, success/failure, model version
- Failed pipeline jobs retried with exponential backoff; dead-letter queue for repeated failures

Design principle: Any stage in the pipeline should be independently replaceable and testable.

Q72: Your AI pipeline has zero visibility into which step is failing. How do you add observability?

Answer:

Adding observability to an opaque pipeline requires instrumentation at every stage without disrupting production.

Step 1 — Structured logging:

- Wrap each pipeline step in try-catch; log: step_name, input_size, output_size, duration_ms, success, error_message

- Correlate via trace_id passed through entire pipeline

Step 2 — Distributed tracing:

- Add OpenTelemetry spans to each step; export to Jaeger/Datadog APM
- Visualize pipeline as waterfall — immediately see which stage takes longest or fails

Step 3 — Metrics:

- Counter: success_count, error_count per step
- Histogram: step_duration_ms per step
- Alert: error rate > 1% on any step

Step 4 — LLM-specific:

- Log prompt + completion (with PII masking) per LLM call
- Log token counts, model version, cost per LLM call

Tools: OpenLLMetry, Langfuse, LangSmith — all provide pipeline-level tracing out of the box.

Result: Any pipeline failure debuggable in < 5 minutes using trace search.

Q73: You quantized your LLM, but accuracy dropped significantly. How do you minimize quantization loss?

Answer:

Significant accuracy drop from quantization is fixable through better quantization techniques and calibration.

Diagnosis:

- Measure perplexity on eval set: quantized vs. base model; > 5% increase signals problematic quantization
- Test specific capability: Is code worse? Math worse? Identify affected domains

Better quantization methods:

- **GPTQ:** Second-order information for minimal quality loss; better than naive INT4.
- **AWQ:** Activation-aware; protects salient weights from quantization error; best INT4 quality.
- **GGUF with Q5_K_M:** Higher bit quantization (5-bit); better quality than Q4.
- **Quantization-Aware Training (QAT):** Train with quantization noise simulation — best quality preservation.

Other techniques:

- Mix precision: Keep critical layers (first/last) in FP16; quantize middle layers to INT4
- Calibration dataset: Use domain-representative data (not generic) for calibration
- Increase bits: Move from INT4 to INT8 if quality is unacceptable

Evaluation: Always evaluate on task-specific benchmark, not just perplexity, before deploying quantized model.

Q74: One failing AI component can take down your entire platform. How do you design graceful degradation?

Answer:

Graceful degradation requires explicit design of failure modes before they happen.

Design principles:

- **Bulkhead isolation:** Each AI feature (recommendation, summarization, chatbot) in separate service; one failure doesn't affect others.
- **Fallback chain per feature:** Every AI feature has a non-AI fallback (rule-based, static, human).
- **Circuit breaker:** Detect failure → open circuit → stop calling failing service → try cached or fallback.

- **Timeout budget:** Each component has a timeout; slow component doesn't make entire request timeout.

Implementation:

- Resilience4j (Java) or Hystrix / Polly (.NET) for circuit breaker + bulkhead patterns
- Feature flags to disable any AI component instantly without deployment
- Health checks per component; orchestrator marks unhealthy and routes around

Runbook per component:

- Define: what degrades, what fallback activates, who is notified, how to recover
- Test: Inject failure quarterly in staging; validate degradation is acceptable

User communication: Display degraded mode notice; don't silently serve worse results without indication.

Evaluation, Testing & Red Teaming

Q75: What is evaluation-driven development for AI applications?

Answer:

Evaluation-driven development (EDD) means defining how you'll measure success before building — then iterating against those metrics.

Process:

- Step 1: Define task and success criteria (what does good look like?)
- Step 2: Build golden dataset of input → expected output pairs
- Step 3: Choose evaluation metrics appropriate to the task
- Step 4: Establish baseline (naive approach or existing system)
- Step 5: Build → evaluate → iterate; only ship when metrics improve vs. baseline
- Step 6: Production monitoring uses same eval metrics as offline

Why it matters:

- Without evals, you iterate on vibes; can't tell if prompt change helped or hurt
- Evals catch regressions before users do
- Forces explicit agreement on what 'better' means across team

Practical:

- Start with 50–100 golden examples; grow to 500+ as system matures
- Include adversarial and edge case examples in eval set
- Run evals on every PR; block merge if eval score drops > 5%

Q76: How do you evaluate LLM outputs? What metrics do you use?

Answer:

LLM evaluation uses a combination of automatic metrics, LLM-as-judge, and human evaluation.

Automatic metrics:

- **BLEU/ROUGE:** N-gram overlap; useful for translation and summarization with reference answers.
- **BERTScore:** Semantic similarity using BERT embeddings; better than BLEU for paraphrase.
- **Exact match:** For factual QA with single correct answer.
- **RAGAS:** RAG-specific metrics: faithfulness, context precision, context recall, answer relevancy.

LLM-as-judge metrics:

- Helpfulness, correctness, safety, conciseness — rated 1–5 by evaluator LLM
- Pairwise comparison: Given Q and two answers, which is better? (reduces scale bias)

Human evaluation:

- Side-by-side comparison of model versions on 100–200 samples
- Structured rubric: Accuracy, completeness, tone, safety — each scored

Task-specific:

- Code: Compilation rate, test pass rate, execution correctness
- QA: Faithfulness (answer grounded in source), answer correctness
- Classification: Precision, recall, F1, confusion matrix

Q77: Explain BLEU, ROUGE, and BERTScore. When would you use each?

Answer:

These three metrics measure different aspects of text similarity between generated and reference text.

- **BLEU (Bilingual Evaluation Understudy):**
- Measures precision of n-gram overlap (generated vs. reference)
- Applies brevity penalty to punish too-short outputs
- Use for: Machine translation (its original purpose); precise wording matters; reference text is gold standard
- Weakness: Doesn't reward paraphrasing; penalizes valid alternative wordings
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):**
- ROUGE-N: N-gram recall (how much of reference appears in output)
- ROUGE-L: Longest common subsequence (word order preserved)
- Use for: Summarization; measures coverage of key points
- Weakness: Same as BLEU — surface-form only; no semantic understanding
- **BERTScore:**
- Uses contextual BERT embeddings to compute semantic similarity
- Robust to paraphrasing; understands synonyms and meaning
- Use for: When paraphrasing is acceptable; complex generation tasks; conversational responses
- Weakness: Computationally heavier; less interpretable

Practical guidance: Use BERTScore as default; supplement with ROUGE-L for summarization. Avoid BLEU except for translation.

Evaluating Large Language Model (LLM) applications—especially Retrieval-Augmented Generation (RAG) systems—requires moving beyond manual spot-checks. Four of the most prominent frameworks for this are **DeepEval**, **RAGAS**, **TruLens**, and **Promptfoo**.

While all four help test model quality, they shine in different parts of the developer workflow.

Quick Comparison

Tool	Primary Focus	Best Used For	Interface
DeepEval	Unit testing for LLMs	Developer CI/CD test automation	Python (pytest)
RAGAS	Component-level RAG metrics	Deep evaluation of retrieval vs.	Python

Tool	Primary Focus	Best Used For	Interface
		generation	
TruLens	The "RAG Triad" + App Observability	Instrumenting and tracking app performance	Python + UI Dashboard
Promptfoo	Prompt & model benchmarking	Fast matrix testing (Prompts × Models)	CLI / YAML / JS

Q78: What is G-Eval, and how does it use LLMs for evaluation?

Answer:

G-Eval is a framework for using LLMs as evaluators of other LLM outputs, with chain-of-thought reasoning.

How it works:

- Step 1: Define evaluation task and criteria (e.g., coherence, relevance, factuality)
- Step 2: Create evaluation prompt with the criteria and chain-of-thought instructions
- Step 3: LLM evaluator reads: task description + output to evaluate + criteria
- Step 4: LLM generates reasoning steps, then assigns a score (1–5)
- Step 5: Optionally, aggregate token probabilities for more granular scoring

Advantages over BLEU/ROUGE:

- Handles open-ended generation where no single reference answer exists
- Can evaluate nuanced qualities (tone, helpfulness, safety) — impossible with n-gram metrics
- Correlates better with human judgment than traditional automatic metrics

Limitations:

- LLM evaluator can be biased (favors verbose responses, self-evaluation bias)
- Expensive: Evaluation cost = generation cost × evaluation cost
- Consistency: Temperature > 0 means repeated evaluation may give different scores

Use with GPT-4 or Claude as evaluator for best correlation with human judgment.

Q79: What is LLM-as-a-judge evaluation, and what are its limitations?

Answer:

LLM-as-a-judge uses a powerful LLM (typically GPT-4/Claude) to evaluate outputs from another LLM, simulating human judgment.

Patterns:

- **Single answer grading:** Judge rates a single response on defined criteria (1–5 scale).
- **Pairwise comparison:** Judge given two responses; pick which is better. Reduces scale inconsistency.
- **Reference-guided:** Judge compares response to gold reference answer for factual tasks.

Advantages:

- Scalable alternative to human evaluation
- Handles open-ended outputs where n-gram metrics fail
- Can evaluate complex qualities: reasoning quality, tone, safety

Limitations:

- **Self-enhancement bias:** LLMs prefer responses similar to their own style; GPT-4 biased toward GPT-4 outputs.
- **Verbosity bias:** LLMs tend to rate longer, more detailed responses higher regardless of quality.
- **Position bias:** In pairwise, first option often preferred (order matters; randomize order).
- **Cost:** Each evaluation call costs money; not free like BLEU.
- **Hallucination:** Judge may hallucinate errors or incorrectly penalize correct responses.

Mitigation: Use pairwise > single grading; randomize option order; cross-validate with human on sample.

Q80: How do you conduct human evaluation for AI systems?

Answer:

Human evaluation is the gold standard — it's expensive but necessary for validating automatic metrics and high-stakes decisions.

Design:

- Define clear rubric: Each criterion (accuracy, helpfulness, safety) has specific scoring guide with examples
- Annotator training: Train annotators on rubric with practice examples; calibrate until inter-rater agreement > 0.7 (Cohen's kappa)
- Blind evaluation: Annotators don't know which model/prompt produced each response
- Sample size: 200–500 examples for statistical significance; power analysis to determine exact N

Evaluation interfaces:

- Labelbox, Scale AI, or custom tool for structured annotation
- Side-by-side for pairwise comparison; single-response for absolute scoring

Quality control:

- Inter-annotator agreement (IAA) measurement; flag low-agreement examples for discussion
- Gold standard questions: Include examples with known correct rating to catch inattentive annotators
- Multiple annotators per item: Use 3 annotators; majority vote or average

Frequency: Run human eval at model launch, after major prompt changes, and quarterly for drift monitoring.

Q81: What is red teaming, and how do you red team an LLM application?

Answer:

Red teaming systematically adversarially tests an LLM application to find safety failures, jailbreaks, and unexpected behaviors before deployment.

Types of attacks to test:

- Prompt injection: Malicious user input overrides system prompt instructions
- Jailbreaks: Adversarial prompts that bypass content policy (DAN, role-play exploits)
- Data extraction: Attempts to extract training data, system prompt, or other users' data
- Hallucination provocation: Prompt designed to elicit confident falsehoods
- Bias elicitation: Prompts designed to surface demographic or political biases
- Goal hijacking: In agentic systems, redirect agent from intended task to malicious action

Process:

- Manual red team: Security researchers + domain experts systematically test over 1–2 weeks
- Automated: LLM-based red team generates adversarial prompts at scale (Garak framework)

- Structured: Define threat model first — who are adversaries? What are high-value attacks?

Output: Report of vulnerabilities with severity ratings; mitigations implemented before launch.

Q82: How do you detect and measure hallucinations in LLM outputs?

Answer:

Hallucination detection measures whether LLM outputs are factually grounded and consistent with provided context.

Types of hallucinations:

- Intrinsic: Contradicts information in the provided context
- Extrinsic: Claims facts not present in context (may or may not be true in world)
- Faithful: Consistent with context but misleading or speculative

Detection methods:

- **NLI-based:** Use a Natural Language Inference model (e.g., DeBERTa-NLI) to check if response is entailed by context.
- **LLM-as-judge:** Ask evaluator LLM 'Is any claim in this response unsupported by the provided context?'
- **SelfCheckGPT:** Generate same response multiple times; inconsistency across generations indicates uncertainty/hallucination.
- **RAGAS Faithfulness:** Specific metric for RAG systems; checks each claim in response against retrieved chunks.

Measurement:

- Hallucination rate: % of responses containing ≥ 1 unsupported claim
- Claim-level: % of individual claims that are unsupported

Mitigation: RAG grounding, chain-of-thought prompting, lower temperature, citation requirements.

Q83: What is adversarial testing for AI systems?

Answer:

Adversarial testing proactively discovers failure modes by testing with inputs specifically designed to break the system.

Categories:

- **Robustness testing:** Input perturbations — typos, paraphrasing, language switches — model should handle gracefully.
- **Boundary testing:** Edge cases at capability limits — ambiguous queries, conflicting instructions.
- **Security testing:** Prompt injection, jailbreaks, data extraction attempts (overlap with red teaming).
- **Fairness testing:** Same question with different demographic descriptors — should produce equivalent answers.
- **Consistency testing:** Semantically identical questions rephrased — should produce consistent answers.

Automated tools:

- Garak: Open-source LLM vulnerability scanner; tests dozens of attack categories
- PromptBench: Adversarial robustness evaluation framework
- LLM-based adversarial generation: Use GPT-4 to generate adversarial prompts for your system

Process: Run before every major release; maintain adversarial test suite in CI; track failure rate by category over time.

Output: Adversarial failure rate by category; severity-ranked findings; mitigations.

Q84: How do you build a regression test suite for AI applications?

Answer:

AI regression tests catch quality degradation when prompts, models, or code change.

Suite composition:

- Golden examples: 200–500 hand-curated input/output pairs representing desired behavior
- Adversarial examples: Known edge cases and previous failure modes that were fixed
- Boundary cases: At-limit inputs (max context, unusual formatting, multiple languages)
- Safety tests: Inputs that should be refused — verify refusal still happens

Metrics per test:

- Exact match (for deterministic outputs)
- BERTScore ≥ threshold (for semantic similarity)
- LLM judge rating ≥ threshold (for complex generation)
- Schema validation pass (for structured output)

CI integration:

- Run on every PR that touches prompt, model config, or retrieval code
- Fail PR if: any safety test fails, or quality score drops > 5% vs. main branch
- Run nightly against production model to catch provider-side model updates

Maintenance: Add new failing cases as tests immediately after fixing them. Suite grows over time.

Q85: What are benchmark suites (MMLU, HumanEval, GSM8K), and how do you interpret them?

Answer:

Benchmark suites are standardized evaluation datasets that enable comparison of LLMs across capabilities.

- **MMLU (Massive Multitask Language Understanding):**
- 57 subjects from STEM, humanities, social science
- Multiple choice questions at undergrad-to-professional level
- Measures: Broad world knowledge and reasoning
- Interpret: 85%+ = expert level; use to compare general knowledge across models
- **HumanEval:**
- 164 Python programming problems with unit tests
- pass@k metric: % of problems solved correctly within k attempts
- Measures: Code generation capability
- Interpret: 85%+ pass@1 = strong coding model; use for code use case model selection
- **GSM8K (Grade School Math):**
- 8,500 grade school math word problems requiring multi-step reasoning
- Measures: Arithmetic reasoning and chain-of-thought ability
- Interpret: 90%+ = strong math reasoning; important for financial/scientific applications

Critical caveat: Models may be trained on benchmark contamination. Benchmarks measure proxy capability — always evaluate on your specific task data before deploying.

Q86: How do you evaluate a RAG system end-to-end?

Answer:

RAG evaluation requires measuring both retrieval quality and generation quality independently and end-to-end.

Retrieval metrics:

- Context Precision: Are retrieved chunks relevant to the query? (precision of retrieval)
- Context Recall: Are all relevant documents retrieved? (recall of retrieval)
- MRR (Mean Reciprocal Rank): How early in the ranked list is the first relevant result?

Generation metrics (given retrieved context):

- Faithfulness: Is the answer grounded in retrieved context? (no hallucination)
- Answer Relevancy: Does the answer address the question?
- Answer Correctness: Is the answer factually correct? (requires ground truth)

Framework: RAGAS (Retrieval Augmented Generation Assessment) provides all these metrics automatically.

End-to-end:

- Golden QA pairs: Questions + expected answers derived from your knowledge base
- Run full pipeline; measure faithfulness, relevancy, correctness
- Component ablation: Swap retriever only, then generator only, to attribute quality to each component

Common failure modes: Low context recall (relevant doc not retrieved), low faithfulness (LLM ignores context).

Q87: How do you evaluate the quality of AI agents?

Answer:

Agent evaluation is harder than single-turn LLM evaluation because quality emerges from multi-step behavior over time.

Dimensions:

- **Task completion rate:** % of tasks successfully completed end-to-end (primary metric).
- **Step accuracy:** Are individual tool calls and decisions correct?
- **Efficiency:** Number of steps taken vs. minimum required; unnecessary steps = low efficiency.
- **Robustness:** Does agent recover from tool failures, unexpected outputs, or ambiguous situations?
- **Safety:** Does agent avoid taking irreversible harmful actions?

Evaluation approaches:

- Environment simulation: Run agent in realistic simulated environment; measure task completion
- Trajectory evaluation: LLM judge evaluates entire action sequence, not just final output
- Checkpoint evaluation: Evaluate agent state at key decision points in workflow
- Human evaluation: Human expert rates agent behavior on realistic tasks

Frameworks: AgentBench, WebArena, ToolBench for standardized agent benchmarks.

Practical: Define 50 representative tasks; track completion rate per task category across model/prompt versions.

Q88: What is the difference between offline and online evaluation for AI systems?

Answer:

Offline and online evaluation serve different purposes and complement each other.

- **Offline evaluation:**
- Conducted before deployment on static, curated datasets

- Controlled environment; same data for every experiment; reproducible
- Fast and cheap iteration — no real users affected
- Metrics: BLEU, BERTScore, RAGAS, human annotation, LLM judge
- Limitation: Distribution shift — eval data may not match production traffic
- **Online evaluation:**
- Conducted in production on real user traffic
- Reflects actual use cases, edge cases, and user behavior
- A/B testing, interleaved experiments, user feedback signals
- Metrics: CSAT, task completion, retention, escalation rate, click-through
- Limitation: Slower; requires significant traffic; user impact from errors

Relationship:

- Use offline eval to screen candidates; only deploy to online eval if offline metrics are acceptable
- Calibrate offline metrics against online metrics — find leading indicators
- Online feedback informs offline eval dataset improvement (close the loop)

Q89: How do you measure factual consistency in LLM outputs?

Answer:

Factual consistency measures whether LLM outputs are internally consistent and consistent with provided source material.

Approaches:

- **NLI-based:** Premise = source document; hypothesis = LLM claim. NLI model classifies as entailment/neutral/contradiction.
- **QA-based (QAFactEval):** Generate questions from source → ask LLM to answer from output → compare answers.
- **Claim extraction + verification:** Extract atomic claims from output → verify each claim against source.
- **LLM verifier:** Ask evaluator LLM to identify claims in output not supported by source.

Tools:

- RAGAS Faithfulness: Ready-made for RAG pipelines
- FActScore: Entity-level factual precision for open-domain generation
- TRUE: Unified framework for factual consistency evaluation

Production monitoring:

- Sample 1–5% of production responses; run automated consistency check
- Alert if consistency score drops below threshold
- Route low-consistency responses to human review queue

Distinguish: Factual consistency (consistent with source) vs. factual accuracy (correct in world knowledge) — different problems.

Q90: How do you evaluate multi-turn conversation quality?

Answer:

Multi-turn evaluation must assess coherence across turns, not just individual response quality.

Dimensions:

- **Turn-level:** Each response quality in isolation (helpfulness, safety, relevance).

- **Coherence:** Does the assistant maintain context correctly across turns?
- **Consistency:** No contradictions between turns (character, facts, stance).
- **Engagement:** Does the conversation flow naturally? Are follow-up questions appropriate?
- **Memory accuracy:** Does assistant correctly recall information mentioned earlier in conversation?

Evaluation methods:

- Holistic LLM judge: Evaluator reads full conversation; rates overall quality
- Turn-by-turn: Rate each turn; track quality degradation over conversation length
- Consistency checker: Specifically test if facts in turn 1 are correctly recalled in turn 10
- Human evaluation: Most reliable; annotators rate full conversations

Metrics: DSTC benchmarks for dialogue; FED (Fine-grained Evaluation of Dialogue) for multi-aspect scoring.

Edge cases: Test long conversations (> 20 turns); ambiguous references; topic switching; corrections.

Q91: What is the role of golden datasets in AI evaluation?

Answer:

Golden datasets are curated ground-truth input/output pairs that serve as the authoritative benchmark for model quality.

Purpose:

- Benchmark: Measure quality of new prompts/models against consistent baseline
- Regression prevention: Catch quality drops when anything changes
- Calibration: Validate that automatic metrics correlate with human judgment
- Communication: Create shared team understanding of what 'good' looks like

Construction:

- Source: Representative sample of real production inputs (not synthetic only)
- Annotation: Human experts provide gold-standard outputs or ratings
- Coverage: Include common cases, edge cases, adversarial cases, failure modes
- Size: 100 examples minimum; 500+ for robust evaluation

Maintenance:

- Version-controlled alongside codebase; update when task evolves
- Periodic refresh: Add new failure patterns; retire outdated examples
- Split: Some examples only used for final evaluation (not shown during development to prevent overfitting)

Pitfall: Golden dataset must not leak into training data — would give artificially high scores.

Q92: How do you implement continuous evaluation for production AI systems?

Answer:

Continuous evaluation detects quality drift and regressions in production before users notice.

Architecture:

- Request sampling: Log 1–5% of production requests (with PII masking)
- Automated scoring: Run BERTScore, RAGAS, or LLM judge on sampled responses nightly
- Feedback integration: Capture explicit (thumbs up/down) and implicit (escalation, repeat query) feedback
- Drift detection: Statistical test (KS test) for distribution shift in input queries
- Alerting: Alert if quality score drops > 5% vs. 7-day rolling average

Evaluation pipeline:

- Nightly batch: Sample → evaluate → store scores in metrics DB → update dashboard
- Real-time: Flag low-confidence responses for immediate human review
- Weekly report: Quality trends, top failure categories, cost per quality point

Triggers for action:

- Score drop > 5%: Investigate prompt/model changes; may indicate provider model update
- Score drop > 15%: Rollback last change; post-mortem
- Score drop > 25%: P0 incident; immediate action

Close the loop: Evaluation findings feed into next sprint's improvements.

Q93: How do you evaluate bias in AI model outputs?

Answer:

Bias evaluation measures whether the model produces systematically different (worse) outputs for some groups vs. others.

Bias types:

- Demographic bias: Different quality/tone for queries about different demographic groups
- Occupational stereotyping: Assumes gender/race for professions
- Sentiment bias: More negative sentiment associated with certain groups
- Performance disparity: Worse accuracy for one demographic's queries than another's

Evaluation methods:

- **Counterfactual testing:** Substitute demographic terms in identical prompts; compare outputs. 'The nurse said... he/she/they' — should be equivalent.
- **Occupational probe:** Generate descriptions of professionals; measure demographic assumptions.
- **Sentiment analysis:** Measure sentiment polarity of outputs across demographic groups.
- **Performance parity:** Measure accuracy on QA tasks across demographic groups; check for disparities.

Frameworks: AI Fairness 360, Fairlearn, Hugging Face evaluate with bias metrics.

Action on finding: Document bias type; mitigate via prompt modification, fine-tuning, or post-processing; re-evaluate.

Ongoing: Run bias evaluation quarterly; track trend; include in model card.

Q94: How do you compare two models or prompts in a statistically rigorous way?

Answer:

Statistical rigor prevents false conclusions from noise — especially important because LLM outputs are variable.

Setup:

- Define primary metric (e.g., win rate in pairwise comparison, or BERTScore)
- Sample size calculation: Power analysis to determine N needed for effect size of interest (typically 200+ pairs)
- Randomize: Randomly assign examples to conditions; control for order effects in pairwise

Statistical tests:

- **Pairwise win rate:** Binomial test or McNemar's test; null hypothesis = 50% win rate.
- **Continuous metrics (BERTScore):** Paired t-test (if normally distributed) or Wilcoxon signed-rank test.

- **Bayesian approach:** Bayesian A/B test gives probability of superiority rather than binary p-value — more actionable.

Corrections:

- Multiple comparisons: If testing 5 metrics, apply Bonferroni correction ($p < 0.01$ per metric)
- Avoid peeking: Don't test for significance after each batch; decide sample size upfront

Reporting:

- Report effect size (Cohen's d) alongside p-value — statistical significance ≠ practical significance
- Confidence interval on win rate: e.g., 'Model A wins 58% ± 4% of the time (95% CI)'

Q95: How do you evaluate the robustness of an LLM application across input variations?

Answer:

Robustness evaluation checks that the system produces consistently high-quality outputs across varied, real-world inputs.

Variation types to test:

- Lexical: Typos, spelling variations, different word choices for same meaning
- Syntactic: Sentence structure variations, passive vs. active voice, question vs. statement form
- Semantic: Paraphrases with identical meaning; should produce equivalent answers
- Length: Short telegraphic queries vs. long detailed queries for same information need
- Language: Non-English inputs; code-mixed inputs; accented spelling
- Format: With/without punctuation; all-caps; markdown vs. plain text

Measurement:

- Generate N variations of each golden example (use LLM for paraphrase generation)
- Score all variations; measure variance — high variance = low robustness
- Consistency rate: % of variations where output quality ≥ baseline threshold

Tools: PromptBench, CheckList (Microsoft), TextFlint for systematic perturbation.

Target: Output quality variance across input variations < 10% — system should be robust to surface-form changes.

Q96: What are the key differences between evaluating traditional ML vs LLM applications?

Answer:

LLM evaluation is fundamentally different from traditional ML evaluation across multiple dimensions.

- **Output type:**
- Traditional ML: Numeric prediction or categorical label → single metric (AUC, RMSE, F1)
- LLM: Open-ended text → requires semantic understanding to evaluate quality
- **Ground truth:**
- Traditional ML: Clear correct answer (price was $X, churn = 1)
- LLM: Often no single correct answer; 'good' is subjective or context-dependent
- **Evaluation cost:**
- Traditional ML: Cheap automated metrics
- LLM: Expensive — requires LLM-as-judge (API cost) or human evaluation
- **Non-determinism:**

- Traditional ML: Deterministic output (no temperature)
- LLM: Same prompt → different outputs; must evaluate distribution, not single output
- **Failure modes:**
- Traditional ML: Prediction error, bias, drift
- LLM: Hallucination, prompt injection, jailbreak, refusal of valid requests, sycophancy

Summary: LLM evaluation requires a fundamentally new toolkit — human eval, LLM judges, semantic metrics, and safety testing — none of which exist in traditional ML evaluation.

Q97: How do you set up an evaluation framework from scratch for a new LLM application?

Answer:

Setting up evaluation from scratch requires a systematic approach before any model work begins.

Step 1 – Define success:

- What does the application do? What does 'good' mean for each output?
- Stakeholder alignment: Product, legal, safety all define quality criteria

Step 2 – Build golden dataset:

- 50 examples minimum; sample from anticipated real traffic distribution
- Include easy cases, edge cases, adversarial cases, and known failure modes

Step 3 – Select metrics:

- Pick 1–2 primary metrics + 2–3 guardrail metrics
- Start with LLM-as-judge for primary; add task-specific metrics as needed

Step 4 – Baseline:

- Evaluate a simple baseline (prompt v0 or existing rule-based system) to establish floor

Step 5 – CI integration:

- Run eval on every PR; block if primary metric drops > 5% or any guardrail fails

Step 6 – Production monitoring:

- Sample production traffic; run same eval nightly; alert on drift

Step 7 – Iteration:

- Eval-driven development: Every change measured against framework; decisions data-driven

---

Q98: Your model passes one fairness metric but fails another. How do you handle conflicting audit results?

Answer:

Conflicting fairness metrics are common because different metrics operationalize fairness differently — and some are mathematically incompatible.

Why metrics conflict:

- Demographic parity (equal positive rate) vs. equalized odds (equal TPR/FPR) are mathematically incompatible when base rates differ (Impossibility theorem)
- Individual fairness vs. group fairness can conflict

How to handle:

- **Step 1: Understand what each metric measures:** Don't pick the metric that passes — understand what each is saying.
- **Step 2: Map to stakeholder values:** Who is harmed by which type of error? Prioritize the metric that minimizes harm to most vulnerable.
- **Step 3: Document the tradeoff:** Be transparent — 'Model achieves X fairness but at cost of Y fairness because of Z.' This is honest engineering.
- **Step 4: Stakeholder decision:** Final choice between conflicting fairness criteria is an ethical/policy decision, not a technical one. Escalate appropriately.
- **Step 5: Mitigate:** Can you improve both metrics together? Resampling, reweighting, threshold adjustment per group.

Principle: No single metric captures all of fairness. Report multiple metrics; be transparent about tradeoffs.

Q99: Your model was fair at deployment, but became biased 6 months later. How do you monitor continuously?

Answer:

Fairness drift happens when population or usage patterns shift, or when the world changes in ways that interact with model behavior.

Causes of fairness drift:

- Population shift: User demographics change over time
- Feedback loops: Model decisions affect outcomes, which affect training data
- World changes: Social/legal changes affect what's appropriate
- Provider model updates: LLM provider silently updates model; behavior changes

Continuous monitoring framework:

- Slice-level metrics: Track quality/outcome metrics per demographic group (age, gender, region) over time
- Fairness dashboard: Weekly automated report of fairness metrics per slice
- Alert thresholds: Alert when any group's metric diverges > 5% from overall metric
- Input drift monitoring: Detect if query distribution changes by demographic group (KL divergence)

Investigation process:

- When drift detected: Compare current vs. 6-month-ago production sample
- Root cause: Is it input drift, model drift, or label drift?
- Remediation: Retrain with current data, adjust prompt, or apply post-processing calibration

Audit cadence: Automated weekly; human fairness review quarterly; external audit annually.

Q100: An external auditor cannot reproduce your model's results. How do you ensure audit reproducibility?

Answer:

Audit reproducibility requires capturing every element that affects model output — model, prompt, parameters, and data.

What must be version-locked:

- Model version: Exact model identifier including provider version (not just 'gpt-4' but 'gpt-4-0613')
- Prompt version: Exact prompt text with version hash
- Inference parameters: Temperature, top_p, max_tokens, seed (use seed=0 for reproducibility)
- Retrieval: If RAG, exact document versions and embedding model version
- Input data: Exact eval dataset with checksums

Implementation:

- Immutable model registry: Model artifacts never modified after tagging
- Prompt registry: Every prompt change creates new version; old versions retained
- Inference config as code: All parameters in version-controlled YAML
- Execution environment: Docker image hash ensures identical software environment
- Audit log: Every production inference logs model_version, prompt_version, parameters, input_hash, output_hash

Auditor package: Provide Docker image + eval dataset + exact configs; auditor can reproduce in isolated environment.

Caveat: LLM providers may not guarantee bit-identical reproducibility even with same seed — document this limitation.

Q101: How do you structure red teaming for an LLM chatbot before launch?

Answer:

Pre-launch red teaming is a structured adversarial assessment that simulates attacks from different adversary profiles.

Phase 1 – Threat modeling (1 week):

- Who are potential adversaries? Malicious users, competitors, curious users, insiders
- What are high-value targets? Data exfiltration, policy bypass, reputational harm, financial fraud
- Prioritize: Risk = likelihood × impact; focus red team effort on high-risk attacks

Phase 2 – Manual red teaming (1–2 weeks):

- Domain experts: Security researchers + subject matter experts for the application domain
- Structured: Each tester assigned attack category (jailbreaks, injection, bias, data extraction)
- Creative: Unstructured free-form adversarial exploration
- Document: Every successful attack → severity (critical/high/medium/low) + reproduction steps

Phase 3 – Automated red teaming:

- Garak framework: Automated LLM vulnerability scanner; runs 100+ attack patterns
- LLM-based attack generation: GPT-4 generates variations of successful manual attacks at scale

Phase 4 – Mitigation & re-test:

- Fix critical/high findings before launch; document accepted risks for medium/low
- Re-test mitigations; verify they don't regress other capabilities

Launch gate: Zero critical findings; < 3 high findings with mitigations in place.

Q102: How do you red team a multimodal model where text-only safety tests miss cross-modal attacks?

Answer:

Cross-modal attacks exploit the interaction between modalities to bypass safety filters designed for single-modality inputs.

Types of cross-modal attacks:

- Text in image: Embed malicious instructions in an image (as visible or near-invisible text); text-only filter misses it
- Image semantics + text instruction: Benign image + malicious text instruction that only makes sense together
- Audio-visual mismatch: In video+audio models, audio carries harmful content while video is benign
- Adversarial images: Visually imperceptible perturbations that steer model behavior
- Homoglyph attacks: Replace text characters with visually similar Unicode characters to bypass text filters

Red teaming approach:

- Dedicated multimodal red team: Testers specifically explore cross-modal attack surfaces, not just single modalities
- Image-based prompt injection: Systematically test images with embedded instructions
- Cross-modal inconsistency: Test cases where modalities give conflicting safety signals
- Automated generation: Use diffusion models to generate adversarial images at scale

Defenses:

- OCR scan on all input images before sending to multimodal model

- Cross-modal consistency check: Safety classifier that evaluates combined input, not each modality separately
- Multi-modal content filter: Train safety classifier on combined modality inputs

RAG Systems & Vector Search

Q63: What is hybrid search, and why is it better than pure vector search?

Answer: Hybrid search combines keyword (BM25) + vector (semantic) retrieval. It is better because it handles exact matches (IDs, names) plus semantic meaning, improves recall and precision, and reduces embedding weaknesses such as those found with rare terms.

Q64: What is re-ranking, and how does it improve RAG retrieval quality?

Answer: Re-ranking is a second-stage model (cross-encoder) that reorders retrieved documents. It improves relevance through better top-k ordering, reduces noise before generation, and boosts answer accuracy.

Q65: How do you handle multi-document and multi-hop questions in RAG?

Answer: To handle these, you should retrieve more documents (higher k), use iterative retrieval (chain-of-thought retrieval), implement chunk linking or citation chaining, use graph-based or agent-based retrieval, and decompose the query into sub-questions.

Q66: What is the "lost in the middle" problem in RAG systems?

Answer: This is a phenomenon where LLMs focus more on the beginning and end of a context window while ignoring middle chunks. Fixes include re-ranking important chunks to the edges, using a sliding window or chunk prioritization, and context compression.

Q67: How do you evaluate a RAG system? Explain faithfulness, relevance, and context precision/recall.

Answer: RAG systems are evaluated using several metrics: **Faithfulness** (is the answer grounded in context?), **Relevance** (does the answer match query intent?), **Context Precision** (% of retrieved docs that are useful), and **Context Recall** (% of needed info actually retrieved).

Q68: Explain Self-RAG. How does the model decide when to retrieve?

Answer: In Self-RAG, the model decides whether to retrieve using reflection tokens or gating. The flow involves the model checking its confidence on a query; if confidence is low, it triggers retrieval, and if high, it answers directly, which reduces unnecessary retrieval calls.

Q69: What is Graph RAG, and when would you use it over traditional RAG?

Answer: Graph RAG uses knowledge graphs (entities and relationships) instead of flat chunks. Use it when multi-hop reasoning is needed, for highly connected data (e.g., research or finance), or when you need explainability via relationships.

Q70: How do you handle structured data (tables, SQL databases) in a RAG pipeline?

Answer: You can convert the query to SQL (text-to-SQL), use hybrid retrieval (vector + DB query), utilize table chunking with schema-aware embeddings, or use specialized tools/agents for execution.

Q71: What are the common failure modes of RAG systems, and how do you debug them?

Answer: Common failures include: **Missing context** (fix: increase recall/better chunking), **Irrelevant docs** (fix: improve embeddings/filtering), **Hallucination** (fix: enforce grounding/citations), **Ranking issues** (fix: add re-ranker), and **Chunk size issues** (fix: tune chunking strategy).

Q72: How do you handle document updates and maintain freshness in a RAG system?

Answer: Strategies include incremental indexing, TTL-based cache invalidation, real-time stream ingestion pipelines, versioned embeddings, and re-embedding only the documents that have changed.

Q73: How do you optimize RAG for latency in production?

Answer: Use smaller embeddings or ANN indexes (FAISS, HNSW), reduce top-k, cache frequent queries, parallelize retrieval and generation, use lightweight re-rankers, or distill the models.

Q74: What is the role of metadata filtering in RAG systems?

Answer: Metadata filtering narrows down documents before retrieval using structured fields (date, source, tags). This results in faster search by reducing the search space, higher relevance, and enables features like personalization and access control.

Q75: What is query transformation in RAG (HyDE, query decomposition, step-back prompting)?

Answer: Query transformation improves retrieval by rewriting queries. **HyDE** generates a hypothetical ideal answer to embed; **Query Decomposition** breaks complex queries into sub-queries; **Step-back Prompting** converts a specific query into a broader conceptual one.

Q76: How do you implement citation and source attribution in RAG?

Answer: Store chunk-to-doc ID mappings, return top-k chunks with metadata (source, page), inject citations during generation via prompts or tools, use structured JSON output with sources, and post-process to align answer spans with sources.

Q77: How do you scale a RAG system to millions of documents?

Answer: Use Approximate Nearest Neighbor (ANN) indexes like FAISS or HNSW, employ sharding and distributed vector DBs, use metadata pre-filtering, implement tiered retrieval (coarse to fine), and utilize caching and query routing.

Q78: What is parent-child chunking, and how does it improve retrieval?

Answer: In this approach, the "parent" is a large context-rich chunk and the "child" is a smaller embedding unit. The system retrieves the child but returns the parent, which improves semantic matching while providing full context.

Q79: Your RAG system is hallucinating despite having the right context. How do you fix it?

Answer: Use a stronger prompt (e.g., “answer ONLY from context”), use extractive QA or constrained decoding, add re-ranking, reduce context noise, and implement an answer verification step or LLM judge.

Q80: Your RAG chunk overlap causes redundant results. How do you reduce redundancy?

Answer: Reduce the chunk overlap size, use Maximal Marginal Relevance (MMR) for retrieval, deduplicate by a similarity threshold, or cluster results and pick representative chunks.

Q81: Your RAG retrieval is too slow with a large knowledge base. How do you speed it up?

Answer: Use ANN instead of brute-force search, lower the embedding dimensions, pre-filter with metadata, cache embeddings and results, and parallelize your queries.

Q82: Your RAG system returns duplicate results. How do you deduplicate?

Answer: Use hash-based or ID-based deduplication, set an embedding similarity threshold, use MMR/diversity-aware retrieval, or perform post-retrieval clustering.

Q83: Your RAG system needs per-user access control on internal documents. How do you implement it?

Answer: Attach Access Control List (ACL) metadata (user/role/org) to documents, filter at query time, use separate indexes per tenant for isolation, and enforce these rules at the retrieval layer rather than the generation layer.

Q84: Your RAG system fails on domain-specific jargon. How do you fix it?

Answer: Fine-tune embeddings on a domain-specific corpus, add a glossary or synonym expansion, use a domain-specific LLM, and implement query expansion.

Q85: Your text-only RAG system now needs to handle images and tables. How do you extend it?

Answer: Use multimodal embeddings like CLIP, implement OCR and table extraction, store structured and unstructured data separately, and use tool-based retrieval such as image captioning or table QA.

Q86: Your RAG knowledge base gets updated frequently and needs versioning. How do you manage it?

Answer: Version your embeddings (using doc_id + version_id), use "soft delete" for old versions, implement incremental re-indexing, and use time-aware retrieval to prioritize the latest information.

Q87: Your RAG system fails on multi-hop questions that require combining multiple facts. How do you fix it?

Answer: Use query decomposition, iterative retrieval (retrieve → reason → retrieve), Graph RAG, and chain-of-thought prompting.

Q88: Your enterprise RAG system returns contradictory answers from different source documents. How do you resolve conflicts?

Answer: Rank sources by trust or recency, show multiple answers with clear citations, use an LLM judge to reconcile the data, and prefer consensus based on majority evidence.

Q89: Your RAG system returns outdated answers from an evolving knowledge base. How do you keep it current?

Answer: Implement time-based filtering, use freshness scoring, establish real-time ingestion pipelines, and perform periodic re-embedding of the knowledge base.

Q90: Your RAG system struggles with PDF documents containing tables and layouts. How do you fix PDF parsing?

Answer: Use layout-aware parsers (like LayoutLM or PDFPlumber), extract tables separately into a structured format, preserve the document's section hierarchy, and chunk by layout rather than just text.

AI Agents & Workflows

Q91: What are the different types of agent memory (short-term, long-term, episodic)? Answer: Short-term memory is the current context or window (task state); **Long-term** is persistent storage like vector DBs; **Episodic** is past experiences and actions plus their outcomes used for learning.

Q92: How do you handle agent failures and implement error recovery?

Answer: Implement retry with backoff, use fallback tools or models, validate outputs before execution, use checkpoints with rollback capabilities, and use error-aware prompting to "fix the previous step".

Q93: What is an agent loop, and how does it decide when to stop?

Answer: An agent loop follows a "think → act → observe → repeat" cycle. It stops when the goal is achieved, max steps are reached, a confidence threshold is met, or an explicit “done” signal is received.

Q94: How do you evaluate and test AI agents?

Answer: Evaluation is based on task success rate, tool accuracy, latency and cost, robustness against edge cases, and human evaluation or simulation tests.

Q95: What are the security risks of agentic systems, and how do you mitigate them?

Answer: Risks include prompt injection, data leakage, and unsafe actions. Mitigation involves input/output sanitization, tool permissioning, sandboxing, rate limits with monitoring, and using allowlisted tools.

Q96: What is the difference between reactive and proactive agents?

Answer: Reactive agents respond only when triggered, while **Proactive** agents anticipate and initiate actions, such as planning or sending alerts.

Q97: How do you manage token consumption and cost in long-running agent workflows?

Answer: Summarize memory, limit the context size, cache responses, use smaller models for simple steps, and set a strict budget per task.

Q98: What is the human-in-the-loop pattern for agents, and when is it needed?

Answer: This involves human approval before critical actions. It is needed for high-risk tasks, ambiguous decisions, and compliance-heavy workflows.

Q99: How do you implement guardrails for AI agents to prevent harmful actions?

Answer: Use rule-based constraints, output validation schemas, policy prompts, tool restrictions, and LLM classifiers specifically for safety.

Q100: What is agent reflection, and how does it improve agent performance?

Answer: Reflection is when an agent reviews its own outputs or errors to improve its next steps, which boosts accuracy and reduces repeated mistakes.

Q101: What is the difference between code-generating agents and tool-calling agents?

Answer: Code-gen agents write and execute code dynamically (flexible but riskier), while **Tool-calling** agents use predefined APIs and tools (safer and more controlled).

Q102: How do you handle multi-modal inputs and outputs in agentic systems?

Answer: Use multimodal models (text+image+audio), convert inputs via OCR or ASR, route tasks to specialized tools, and use unified embeddings.

Q103: How do you implement state management in complex agent workflows?

Answer: Use an external state store like Redis or a DB, track every step and tool output, use structured workflows (FSM/DAG), and persist checkpoints.

Q104: How do you build a customer support agent with escalation logic?

Answer: Use intent detection for simple queries, set a confidence threshold for escalation, use sentiment analysis to escalate negative interactions, and ensure a human handoff includes full context.

Q105: What is agent orchestration, and how do you implement it?

Answer: Orchestration is managing multiple agents and tools in workflows. It is implemented using planners and executors, DAG pipelines, and frameworks like LangGraph or CrewAI.

Q106: How do you build a code execution agent safely using sandboxed environments?

Answer: Use sandboxes like Docker or VMs, set resource limits (CPU/memory), restrict network and file system access, and validate code before running it.

Q107: Your AI agent is stuck in an infinite loop. How do you detect and break the cycle?

Answer: Set a max iteration cap, detect repeated actions, use no-progress heuristics, and implement a timeout kill-switch.

Q108: Your AI agent gets conflicting answers from different tools. How does it reconcile them?

Answer: Rank tools by trust or confidence, cross-check with a third tool, ask the LLM to reconcile the differences, or show multiple answers if necessary.

Q109: Your AI agent keeps exceeding its budget per task. How do you enforce budget limits?

Answer: Track tokens and cost per task, set hard limits that stop execution, use adaptive model selection (cheaper models), and implement early stopping.

Q110: Your AI agent hallucinates tool capabilities and passes wrong inputs. How do you fix it?

Answer: Use strict JSON schemas for tools, use native function calling, implement validation with retries, and provide few-shot examples in the prompt.

Q111: Your AI agent deleted a production database. How do you prevent irreversible actions?

Answer: Implement human-in-the-loop confirmation steps, use read-only defaults, maintain audit logs, set up rollback mechanisms, and use permission gating.

Q112: Your AI agent has many tools, but keeps picking the wrong one. How do you improve tool selection?

Answer: Provide better tool descriptions, use specialized tool routing models, include few-shot examples, and reduce the tool set available for specific tasks.

Q113: Your AI agent takes too long to complete a task. How do you speed it up?

Answer: Parallelize tool calls, cache results, reduce the number of steps, use faster models, and pre-fetch necessary data.

Q114: Your LLM selects the right tool but extracts the wrong parameters. How do you fix parameter extraction?

Answer: Enforce structured outputs (JSON schema), use validation with retries, use extraction-specific prompts, or use fine-tuning and few-shot examples.

Fine-Tuning & Model Adaptation

Q115: What is fine-tuning, and when should you fine-tune an LLM?

Answer: Fine-tuning is training an LLM on domain-specific or task-specific data. Use it when there are repeated patterns, for domain adaptation, or when prompting/RAG is insufficient.

Q116: Explain the difference between full fine-tuning and parameter-efficient fine-tuning (PEFT). Answer: Full FT updates all weights (high cost, high control); **PEFT** updates only a small set of parameters (cheaper, faster, and more scalable).

Q117: What is LoRA (Low-Rank Adaptation), and how does it work?

Answer: LoRA adds low-rank matrices to weights ($\Delta W = A \times B$). It works by freezing the base model and training only the small rank layers, making it efficient and modular.

Q118: What is QLoRA, and how does it enable fine-tuning on consumer hardware?

Answer: QLoRA quantizes the base model to 4-bit and then applies LoRA on top. This allows large models to fit on consumer-grade GPUs.

Q119: Explain Prefix Tuning and Prompt Tuning. How are they different from LoRA? Answer: Prompt tuning learns soft tokens at the input; **Prefix tuning** injects tokens into all layers. **LoRA** is different because it actually modifies the weights, making it more expressive.

Q120: What is adapter-based fine-tuning?

Answer: This involves inserting small trainable modules between layers. It is similar to LoRA but uses explicit modules that can be swapped per task.

Q121: What is RLHF (Reinforcement Learning from Human Feedback), and how is it used to align LLMs?

Answer: RLHF aligns models with human intent using a three-step process: SFT → training a reward model → RL optimization (PPO) based on human preferences.

Q122: What is instruction tuning, and why is it important for chat models?

Answer: It is training on (instruction → response) pairs. It is important because it enables the model to follow natural language commands effectively.

Q123: How do you prepare a dataset for fine-tuning an LLM?

Answer: Clean and deduplicate data, format it (instruction, input, output), balance the classes/tasks, and ensure high-quality labels.

Q124: What is catastrophic forgetting, and how do you prevent it during fine-tuning?

Answer: This is when a model forgets general knowledge after fine-tuning. Prevention includes mixing general and domain data, using PEFT, and using a lower learning rate.

Q125: When should you choose fine-tuning over RAG over prompt engineering? Answer: Prompting is for quick tasks with no infra; **RAG** is for dynamic knowledge; **Fine-tuning** is for learning specific behaviors, styles, or deep domain adaptation.

Q126: How do you evaluate a fine-tuned model's performance?

Answer: Use task metrics (accuracy, F1), human evaluation, benchmark datasets, and regression tests.

Q127: What is synthetic data generation, and how do you use it for fine-tuning?

Answer: It involves using LLMs to generate training data to augment low-data domains, followed by filtering for quality.

Q128: What are the key hyperparameters for fine-tuning (learning rate, epochs, batch size, LoRA rank)?

Answer: The learning rate is the most critical. Others include epochs, batch size, and LoRA rank (which balances capacity vs. cost).

Q129: How do you fine-tune a model for a specific domain (legal, medical, finance)?

Answer: Use a domain-specific corpus, add terminology datasets, validate results with experts, and apply safety constraints.

Q130: What is continual pre-training, and when would you use it?

Answer: It is training on a large domain corpus before fine-tuning. Use it when the domain shift is very large.

Q131: How do you merge multiple LoRA adapters?

Answer: Through weighted merging, sequential composition, and resolving conflicts via evaluation.

Q132: What is the difference between SFT (Supervised Fine-Tuning) and alignment training? Answer: SFT is supervised learning using specific labels; **Alignment** uses RLHF/RLAIF to optimize based on preferences.

Q133: What is RLAIF (RL from AI Feedback), and how does it differ from RLHF?

Answer: RLAIF uses AI feedback instead of humans. It is more scalable and cheaper but can be less reliable.

Q134: What is knowledge distillation for fine-tuning, and what are the legal considerations?

Answer: It's training a small model from a large model's outputs. Legal concerns include data ownership, output licensing, and avoiding the copying of proprietary content.

Q135: Your fine-tuned LLM produces factually wrong outputs due to training data quality issues. How do you fix it?

Answer: Clean the dataset, remove noisy labels, add high-quality data, and use validation checks.

Q136: You must choose between LoRA and full fine-tuning for a domain-specific assistant. How do you decide?

Answer: Use **LoRA** if you have limited compute or multiple tasks; use **Full FT** if you have large datasets and need deep architectural changes.

Q137: Your fine-tuned model memorized training data verbatim instead of learning patterns. How do you fix overfitting?

Answer: Use more data, apply regularization, use early stopping, and perform data augmentation.

Q138: Your fine-tuned LLM forgot its general capabilities after domain-specific fine-tuning. How do you fix catastrophic forgetting?

Answer: Use mixed training (general + domain), lower the learning rate, or use PEFT (LoRA/adapters).

Q139: Your RLHF preference data has low annotator agreement. How do you ensure data quality?

Answer: Improve the guidelines, filter out low-agreement samples, use expert annotators, and measure inter-annotator agreement scores.

Vector Databases & Embeddings

Q140: What are embeddings in the context of AI engineering?

Answer: They are numerical vector representations of data (text, image, etc.) that capture semantic meaning.

Q141: How do embedding models convert text to vectors?

Answer: They tokenize text, pass it through a transformer, and extract hidden states (CLS / pooled output) to create a dense vector.

Q142: What is the difference between sparse and dense embeddings? Answer: Sparse vectors are mostly zeros (e.g., TF-IDF, BM25); **Dense** vectors contain continuous values for semantic meaning.

Q143: Explain cosine similarity, dot product, and Euclidean distance for vector search. Answer: Cosine measures angle similarity (most common); **Dot product** measures magnitude and direction; **Euclidean** measures physical distance in space.

Q144: What is a vector database, and how does it differ from a traditional database?

Answer: A vector DB is built for similarity search (ANN), whereas a traditional DB is built for exact match queries.

Q145: How do you choose the right embedding model for your use case?

Answer: Consider domain relevance, benchmark scores (MTEB), latency, cost, and multilingual requirements.

Q146: What is embedding dimensionality, and how does it affect performance and cost?

Answer: It is the vector size (e.g., 768 or 1536). Higher dimensionality increases accuracy but adds cost; lower is faster and cheaper.

Q147: How do you handle embedding drift when the embedding model is updated?

Answer: Version your embeddings, re-index gradually, A/B test the models, and maintain backward compatibility.

Q148: What are multi-modal embeddings, and how are they generated?

Answer: These are a single vector space for different types of data; models like CLIP align these modalities.

Q149: How do you index and query multi-tenant data in a vector database?

Answer: Use a namespace per tenant, apply metadata filtering, or use completely separate indexes for strict isolation.

Q150: What is quantization of embeddings, and how does it reduce storage costs?

Answer: It reduces precision (e.g., float32 to int8), which uses less memory and allows for faster search.

Q151: How do you benchmark and evaluate embedding model quality?

Answer: Use retrieval metrics (Recall@k, MRR), semantic similarity tests, and measure downstream task performance.

Q152: What is the role of metadata in vector databases?

Answer: It is used for filtering (date, type), improves relevance, and enables hybrid queries.

Q153: How do you handle large-scale vector search with billions of vectors?

Answer: Use ANN indexes (HNSW, IVF), employ sharding and distributed systems, and utilize GPU acceleration.

Q154: What is hybrid search (combining keyword search with vector search)?

Answer: It combines BM25 keyword matching with vector search to get the best of exact and semantic matching.

Q155: How do you fine-tune an embedding model for a specific domain?

Answer: Use contrastive learning with positive/negative pairs and domain-specific datasets.

Q156: Your vector database for RAG is consuming too much memory. How do you reduce it?

Answer: Implement quantization, reduce the dimensions, deduplicate vectors, and use IVF or Product Quantization (PQ).

Q157: Your vector database cannot scale to millions of embeddings. How do you fix the bottleneck?

Answer: Use ANN (HNSW, IVF), partition or shard the data, and move to a distributed vector DB.

Q158: Your new embedding model has different dimensions from the existing vectors in production. How do you handle the mismatch?

Answer: Re-embed all your data, maintain separate indexes for different versions, and use versioned embeddings.

Q159: Your vector search returns irrelevant results despite high similarity scores. How do you fix it?

Answer: Switch to a better embedding model, add a re-ranking step, improve your chunking strategy, or use hybrid search.

Q160: You deployed a new embedding model, and search quality crashed overnight. How do you handle embedding drift?

Answer: Roll back the model immediately, ensure you A/B test before deployment, perform a gradual migration, and monitor metrics closely.

Q161: Your semantic search fails for short queries. How do you improve it?

Answer: Use query expansion, hybrid search (with a BM25 boost), fine-tune the embeddings, or add more context to the query.

Safety, Ethics & Privacy

Q162: What are hallucinations in LLMs, and how do you mitigate them?

Answer: These are incorrect or fabricated outputs. Mitigation includes RAG grounding, better prompts, citations, verification models, and fine-tuning.

Q163: What is prompt injection, and what are the different types (direct, indirect)?

Answer: It is malicious input designed to manipulate the model's behavior. **Direct** is a user input attack; **Indirect** is hidden in retrieved data or web pages. Mitigation involves input sanitization, tool isolation, and allowlists.

Q164: How do you implement input and output guardrails for AI systems? Answer: Input guardrails use validation, filtering, and classification; **Output guardrails** use moderation models, schema checks, and safety filters.

Q165: What is AI alignment, and why is it important?

Answer: Alignment ensures model behavior matches human values. It is critical for safety, trust, and compliance.

Q166: How do you detect and mitigate bias in AI systems?

Answer: Measure fairness metrics, use balanced datasets, debias the training process, and apply post-processing corrections.

Q167: What are the key data privacy considerations (GDPR, CCPA) when building AI applications?

Answer: Key factors include user consent, purpose limitation, data minimization, the right to access/delete data, and secure storage.

Q168: How do you handle PII in LLM inputs and outputs?

Answer: Detect it with Named Entity Recognition (NER) models, mask or anonymize it, avoid storing raw PII, and encrypt all sensitive data.

Q169: What is explainability in AI, and why does it matter?

Answer: It is the ability to justify decisions, which is vital for building trust and ensuring compliance.

Q170: What is the difference between interpretability and explainability? Answer: Interpretability refers to model transparency; **Explainability** refers to providing human-understandable reasons for an output.

Q171: How do you build trust with users in AI-powered applications?

Answer: Through transparency, providing citations, maintaining consistent behavior, and offering a human fallback.

Q172: What are adversarial attacks on AI systems, and how do you defend against them?

Answer: These are inputs crafted specifically to break models. Defense includes robust training, rigorous validation, and continuous monitoring.

Q173: What is data poisoning, and how can it affect AI models?

Answer: It is the corruption of training data, which leads to bad model behavior. Mitigation involves data validation and anomaly detection.

Q174: How do you implement content safety filters for AI-generated content?

Answer: Use classification models, rule-based filters, and human review loops.

Q175: What is responsible AI, and what frameworks exist for implementing it?

Answer: It refers to ethical, fair, and safe AI usage. Frameworks include the NIST AI RMF and OECD AI Principles.

Q176: How do you handle copyright and intellectual property concerns with AI-generated content?

Answer: Avoid verbatim reproduction, track sources, and use only licensed data.

Q177: What is the EU AI Act, and how does it affect AI engineering?

Answer: It is a risk-based regulation that imposes strict rules for high-risk AI systems.

Q178: How do you implement audit trails and logging for AI decisions?

Answer: Log all inputs, outputs, and decisions; store metadata and timestamps; and ensure full traceability.

Q179: What is model card documentation, and why is it important?

Answer: It is documentation of model behavior, risks, and limitations, which ensures transparency and governance.

Q180: How do you handle misuse and abuse of AI systems in production?

Answer: Use abuse detection systems, implement rate limiting, and maintain account-level controls.

Q181: What is differential privacy, and how can it be applied during model training?

Answer: It adds noise to protect individuals, creating a tradeoff between privacy and accuracy.

Q182: How would you design an AI incident response plan?

Answer: Follow a "Detect → Contain → Investigate → Fix → Communicate" flow, followed by a post-mortem and prevention plan.

Q183: What is the NIST AI Risk Management Framework (AI RMF)?

Answer: It is a framework for implementing responsible AI.

Q184: Your healthcare chatbot gives medical diagnoses it should not make. How do you add safety guardrails?

Answer: Restrict the model from making diagnoses, add clear disclaimers, and route medical queries to human experts.

Q185: Your AI system is reproducing copyrighted material verbatim. How do you prevent this?

Answer: Deduplicate training data, perform output similarity checks, and refuse to output long verbatim segments.

Q186: Your resume screening AI rejects more female candidates for engineering roles. How do you fix gender bias?

Answer: Rebalance your data, remove biased features, and apply fairness constraints.

Q187: Your AI model passes bias checks by gender and race separately, but fails for intersectional groups. How do you handle it?

Answer: Evaluate combined groups specifically and use subgroup fairness metrics.

Q188: Your AI denied a loan, and the customer demands a GDPR explanation. How do you provide one?

Answer: Provide a feature-level explanation and use transparent reasoning (avoiding pure "black-box" decisions).

Q189: A user invokes the right to be forgotten, but their data is in your model weights. How do you comply?

Answer: Delete the user data, retrain the model or use machine unlearning, and avoid storing sensitive data in weights originally.

Q190: The EU AI Act may classify your AI system as high-risk. How do you comply?

Answer: Perform a risk assessment, create full documentation, ensure human oversight, and implement continuous monitoring.

Q191: Your differentially private model lost significant accuracy. How do you balance privacy and utility?

Answer: Tune the privacy budget (ϵ), use hybrid approaches, and apply DP only where strictly needed.

Q192: One malicious participant is poisoning your federated learning model. How do you defend against it?

Answer: Use robust aggregation methods (like median or Krum), detect anomalies, and use reputation scoring.

Q193: Your AI hiring model uses proxy features for protected attributes. How do you eliminate proxy discrimination?

Answer: Detect features correlated with protected attributes and either remove or regularize those proxies.

Q194: Your predictive model creates a feedback loop of biased outcomes. How do you break it?

Answer: Introduce randomness, perform periodic retraining, and apply fairness constraints.

Q195: Your AI generates fake news images. How do you implement watermarking for AI-generated content?

Answer: Embed invisible signals and use systems to detect synthetic media.

Q196: Your AI denies a service, and the user has no way to challenge it. How do you design an appeals process?

Answer: Provide a user feedback channel, implement human review, and ensure a transparent re-evaluation process.

Q197: An auditor asks why your AI rejected a request 6 months ago, and you have no logs. How do you build audit trails?

Answer: Implement structured logging, set clear retention policies, and maintain immutable audit logs.

Q198: You removed PII, but users were re-identified from anonymized data. How do you prevent re-identification?

Answer: Use strong anonymization techniques like k-anonymity or differential privacy, and limit data linkage capabilities.

Q199: A pre-trained model from an open-source repo may contain a hidden backdoor. How do you detect it?

Answer: Test with triggers, analyze the model for unusual behavior, and perform fine-tuning to remove potential backdoors.

Q200: Your LLM's training data was deliberately poisoned by an adversary. How do you respond?

Answer: Identify the bad samples, retrain the model, and strengthen your data pipeline security.

Q201: Your AI mental health chatbot gave harmful advice to a user in crisis. How do you mitigate harm?

Answer: Use crisis detection, immediately escalate to human services, and use safe response templates.

Q202: Your AI system caused incorrect critical decisions. How do you run a blameless post-mortem?

Answer: Focus on system failures rather than individuals, document the root cause, and create a plan to prevent recurrence.

Q203: Radiologists agree with AI 98% of the time, even when it is wrong. How do you prevent human over-reliance on AI?

Answer: Show model uncertainty, encourage verification, and maintain human-in-the-loop workflows.

Q204: Your content moderation flags normal cultural expressions as offensive in other markets. How do you adapt cross-culturally?

Answer: Use localized models, cultural datasets, and region-specific rules.

Q205: Your AI training produces massive carbon emissions. How do you reduce environmental impact?

Answer: Use efficient models (like distillation), use green data centers, and optimize the training process.

Q206: How would you build a sentiment classifier to classify text documents into three classes: negative, neutral, and positive? Also, how would you extract the parts of the text that contributed to the predicted class? Start with the simplest method and progress to your most complex method.

Answer: Build a sentiment classifier to classify text into negative, neutral, and positive classes and extract contributing text parts.

Approach

Start with a simple rule-based approach using keyword dictionaries for negative, neutral, and positive words.

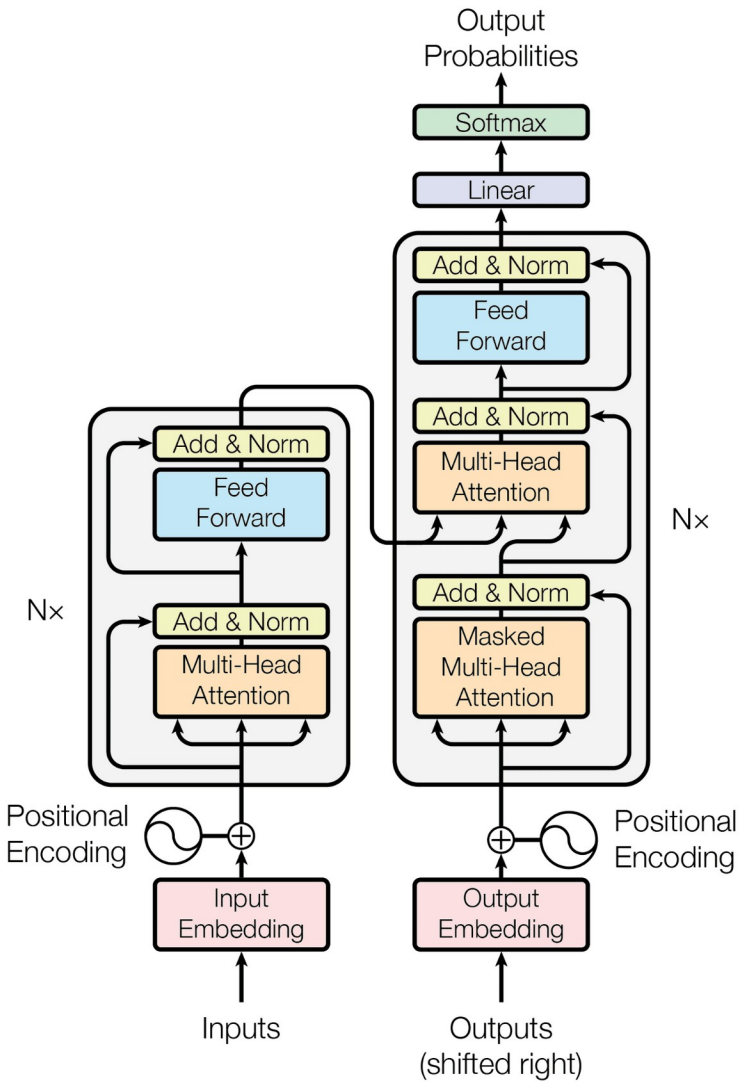
Use a classical machine learning approach: preprocess text, convert to features using TF-IDF, and train a classifier like Logistic Regression.

Transformer Architecture

Tokenization & Embeddings: Input text is broken into smaller units (tokens) and converted into numerical vectors.

Positional Encoding: Because the model processes all data in parallel, it needs a way to know the order of the words. Positional encodings are injected into the vectors to add sequence information.
Self-Attention: This layer calculates Query, Key, and Value (QKV) matrices for each token. It determines which words in a prompt require the most focus (e.g., in "The bank of the river," the attention layer associates "bank" with "river" instead of "money").

Encoder and Decoder: The original model features an encoder that processes the input and a decoder that generates the final output. Many modern LLMs (like GPT) utilize a decoder-only structure designed specifically for text generation.



Move to a deep learning approach with pretrained transformer models (e.g., BERT) fine-tuned on labeled sentiment datasets for better accuracy. For interpretability, extract parts of the text contributing to classification by using:

- For rule-based, return matched keywords.

- For classical ML, use feature importance or coefficients to highlight influential words.

- For deep learning, use attention weights or explanation methods like LIME/SHAP to highlight key text fragments.

Q207: In the attention mechanism, what causes the matrices to attend to relevant parts of the input sequence? Why are they not the same, and how can they differ for each sample of each batch?

Answer: Mechanism: Attention computes relevance via dot-products between Query (Q) and Key (K) vectors and normalizes with softmax: attention = softmax(Q K^T / sqrt(d)) · V. The dot product measures similarity in the learned representation space, so high similarity → more attention weight.

Why they attend to relevant parts: Learned linear projections (weight matrices) map raw token embeddings into Q/K/V spaces where geometric proximity encodes relevance; training shapes these projections so tokens that should interact end up with higher dot-products.

Why Q, K, V are not the same: They are produced by different learned projection matrices (different parameters) so each plays a distinct role — Q asks “what I need”, K says “what each position offers”, V supplies the actual content. Using different projections lets the model separate roles and capture richer interactions.

Per-sample / per-batch variation: Q/K/V depend on the input embeddings and positional encodings, so different samples produce different Q/K/V even with shared weights. Softmax is applied per sample and per head, giving unique attention distributions each time.

Multi-heads and positional/contextual effects: Multiple heads use different projection subspaces, enabling diverse attention patterns. Positional encodings and previous-layer context further change Q/K/V across tokens and samples, so attention differs for each example.

Normalization & training dynamics: The sqrt(d) scaling and softmax stabilize gradients; training updates projection weights to align Q/K similarity with task-specific relevance, making attention focus both learned and input-dependent.

Q208: How would you fully automate training in AWS Sagemaker?

Answer: Overview: Build an end-to-end automated pipeline that covers data ingestion → preprocessing → training → evaluation → model registry → deployment → monitoring, using SageMaker-native services and IaC for repeatability and governance.

Orchestration: SageMaker Pipelines (or Step Functions) to define reusable steps: Processing jobs, Training jobs, Hyperparameter Tuning, Evaluation, and Model Registration; trigger pipelines via EventBridge on data arrival or on a schedule.

Data & Feature Management: Store raw and processed data in S3, use SageMaker Processing / Feature Store for transformations and feature persistence, and enforce schema/validation tests in a processing step.

Training & Tuning: Use SageMaker Training with containerized training scripts (Script Mode) or built-in estimators; run Hyperparameter Tuning Jobs and distributed training as needed; capture metrics with CloudWatch and ML Metadata in Pipeline step outputs.

Model Registry & CI/CD: Register models in SageMaker Model Registry with approval workflows; implement CI/CD using CodePipeline/CodeBuild or GitHub Actions integrated with Pipelines to promote models from staging to production after automated checks.

Security & Infra: Manage access with IAM, encrypt artifacts with KMS, and provision resources via CloudFormation/Terraform to ensure reproducible environments and compliance.

Monitoring & Retraining: Deploy model endpoints (SageMaker Endpoints or Batch Transform) and enable Model Monitor for data/feature drift, alerting, and automated retraining triggers that re-run the pipeline when performance drops below thresholds.

Q209: After you have trained or fine-tuned a transformers model, what are some steps you would take to make the inference faster?

Answer:

Quantization: Convert weights to INT8 or FP16 using post-training or quant-aware methods to reduce memory and speed up compute on supported hardware.

Distillation & pruning: Use knowledge distillation to train a smaller student model and apply structured/unstructured pruning to remove redundant parameters while maintaining accuracy.

Model architecture & optimization: Replace heavy blocks with efficient variants (e.g., ALBERT, TinyBERT, or Linformer) or reduce layers/hidden size; use attention sparsity or efficient attention implementations.

Compilation & runtime: Export to ONNX/TorchScript and run on optimized runtimes (TensorRT, OpenVINO, TVM) to get kernel-level speedups and better memory usage.

Inference techniques: Enable mixed precision, cache key/value states for autoregressive decoding, use dynamic or grouped batching, and limit max sequence length or use token pruning.

Deployment considerations: Choose appropriate hardware (GPU/TPU/accelerator), tune batch size and concurrency, and monitor latency vs. throughput trade-offs with profiling to iterate.

Q210: What are L1 and L2 regularization, and how do they help prevent overfitting? Explain the differences between them and when you might prefer one over the other.

Answer: Definition: L1 regularization adds the sum of absolute weights to the loss (loss + λ * sum |w_i|). L2 regularization adds the sum of squared weights (loss + λ * sum w_i^2). Both penalize large weights to reduce model complexity and overfitting.

How they prevent overfitting: By shrinking weights, they reduce variance and force simpler models that generalize better. The regularization hyperparameter λ controls the strength of the penalty.

Effect on weights:

L1: encourages sparsity; many weights become exactly zero → feature selection.

L2: encourages small but nonzero weights; distributes penalty across correlated features.

Optimization behavior:

L1: non-differentiable at zero, often solved with subgradients; yields sparse solutions.

L2: smooth quadratic penalty, yields closed-form shrinkage (ridge), numerically stable.

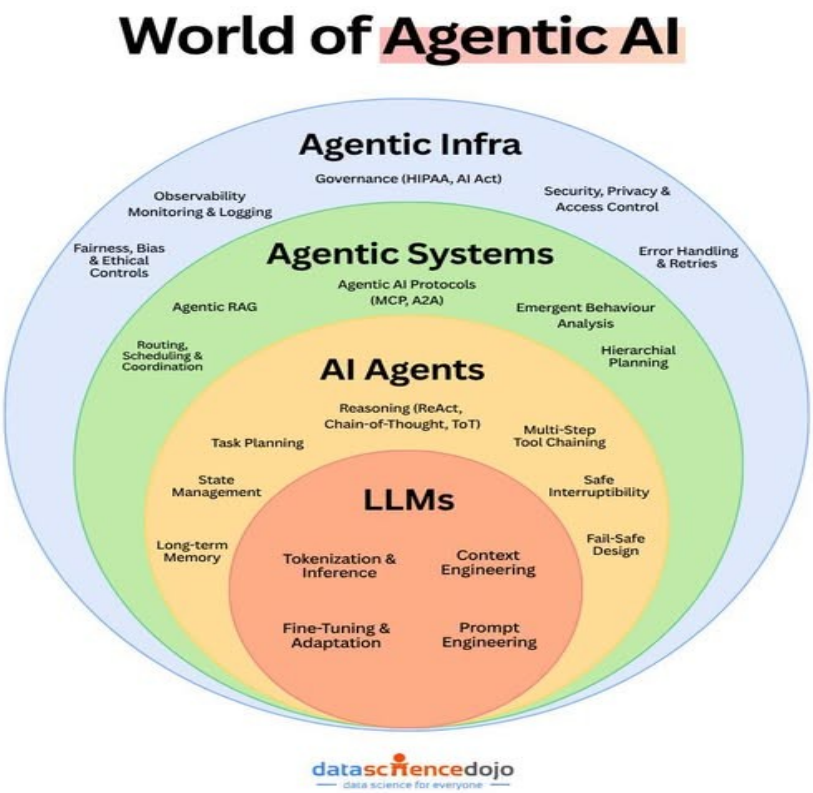
When to prefer which:

Prefer L1 when you expect many irrelevant features and want automatic feature selection.

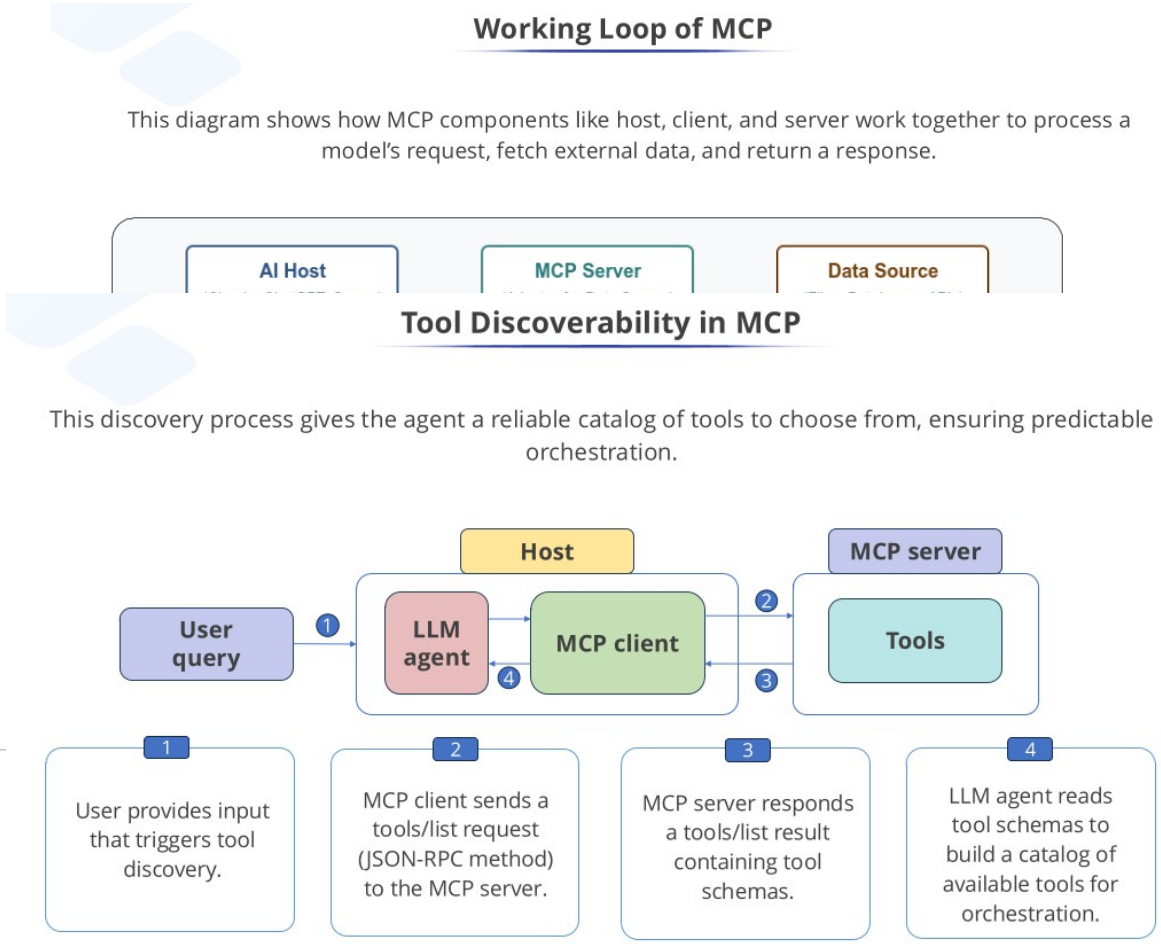
Prefer L2 when features are many but informative, or when you want stable, small weights (helps with multicollinearity).

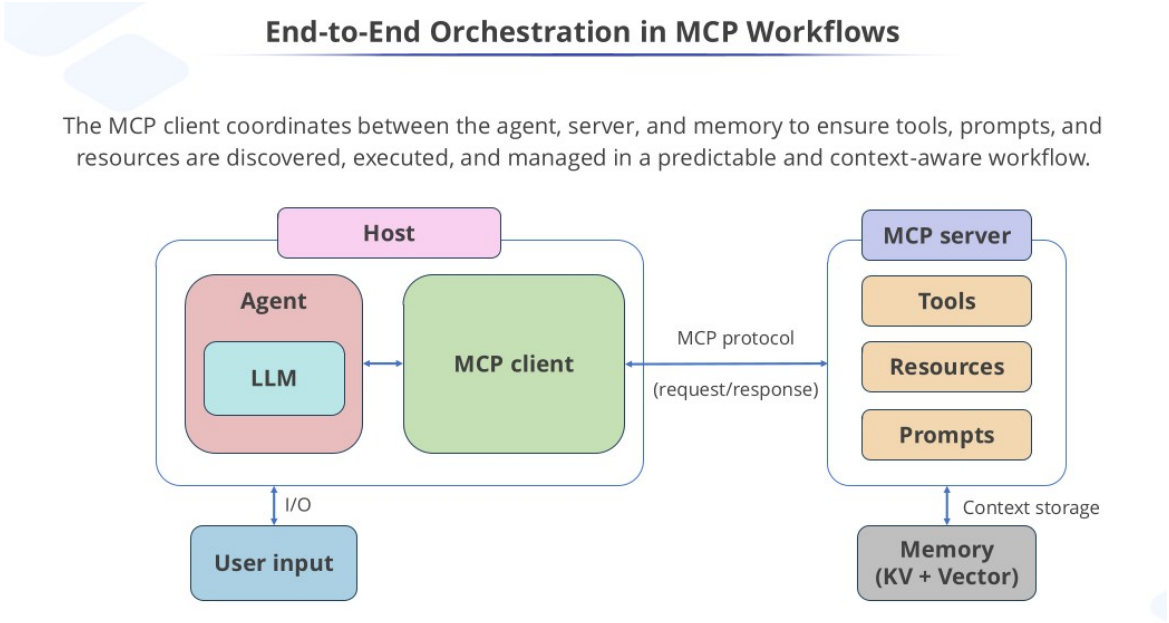
Use Elastic Net (combined L1+L2) when you want both sparsity and grouping of correlated features.

Practical note: tune λ via cross-validation; scale features (standardize) before applying regularization for consistent effect.



How MCP [Model Context Protocol] work





To deploy an Agentic AI application on Azure, utilize Azure Container Apps alongside the Microsoft Agent Framework (MAF) and Azure AI Foundry. This stack provides the scalable microservices architecture needed for autonomous agents that execute multi-step tool calls, while Foundry provides the central control plane for monitoring and observability.

Step-by-Step Deployment Guide

1. Set Up Your Foundation and Models

- Create a project workspace within Azure AI Foundry.
- Navigate to the Model Catalog and deploy a large language model (e.g., ,) or an open-source model optimized for agents.

2. Package Your Agent as a Container

- Build your agent's logic (incorporating tools like Bing Search or Azure AI Search) using the **Microsoft Agent Framework**.
- Containerize your application with Docker. Ensure your exposes the required port (e.g., port 80 or 8080) for the agent's invoke endpoint.

3. Deploy to Azure Container Apps (ACA)

- Use the Azure Developer CLI () to provision and deploy. Run the following command in your terminal from your project directory:

- Once deployment completes, the ACA will output your **Application URL**.

4. Connect to Azure AI Foundry for Observability

- After deploying, navigate to the **Monitoring** tab in your Azure AI Foundry dashboard.
- Connect the newly created Application Insights resource to your Foundry project to monitor your agent's reasoning, planning, and tool calls.

5. Test Your Deployment

- Save the URL from your container app to an environment variable in your terminal, and test your agent by sending a POST request.

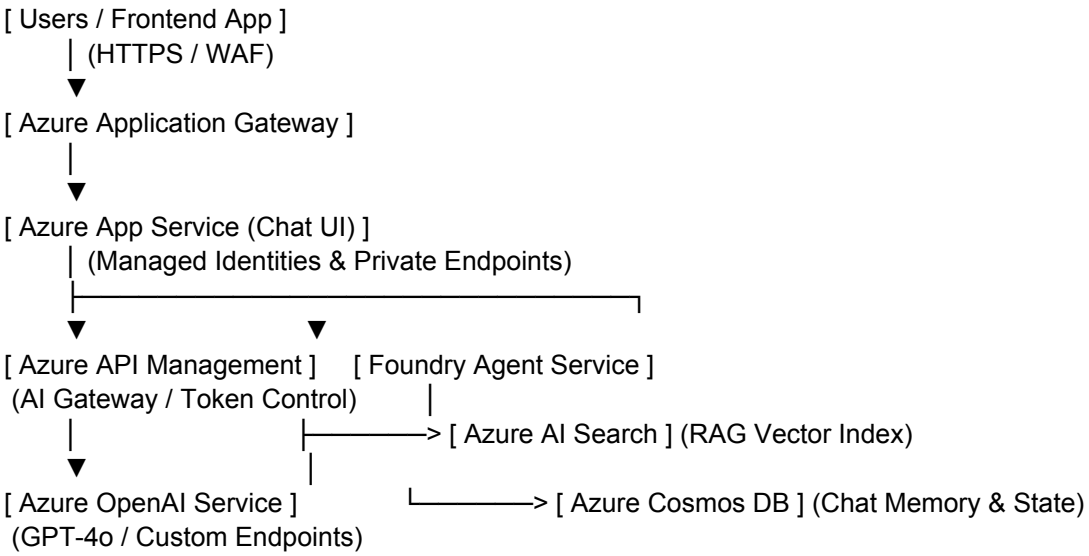
Tell me Azure AI and Agentic AI Services you used?

- Azue AI foundry
- Agent Services
- AI Serach
- OpenAI Services
- Cosmos DB
- Azure SQL
- App services
- Azure Developer CLI (azd)
- [Your Server / DB] ──(ExpressRoute/VPN)──> [Azure Private Endpoint] │ ▼ [Azure App Service] <──> [Azure AI Agent Service] <──> [Azure AI Search] (Python FastAPI) (AutoGen / Python SDK) (Hybrid Vector DB)

Azure Scalable AI Architecture (The Microsoft Foundry Pattern)

This pattern uses Azure's native enterprise ecosystem, leaning heavily on Azure OpenAI, Microsoft Foundry, and Azure AI Search for secure, isolated document grounding.

Plaintext



Key Components & Implementation Details

1. **Perimeter & Traffic Management:**

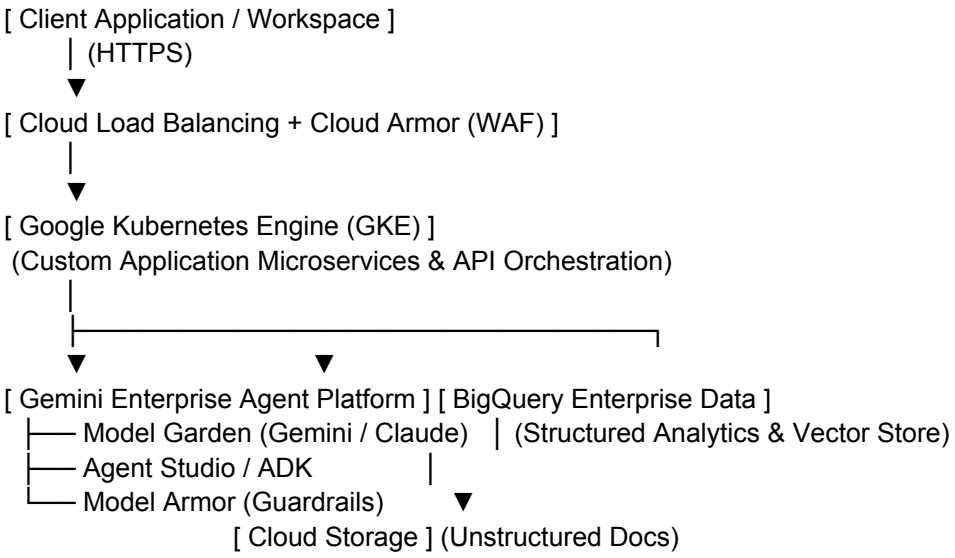
- External users hit an **Azure Application Gateway** integrated with a Web Application Firewall (WAF) to block malicious traffic.
- Traffic routes to an **Azure App Service** hosting the user interface or orchestration microservices.

2. **The AI Gateway:**
    - Instead of apps calling models directly, traffic flows through **Azure API Management (APM)** configured as an AI Gateway. APM handles enterprise quotas, global load balancing across model regions, semantic caching, and unified token-spend logging.
3. **Orchestration & Agents:**
    - **Microsoft Foundry Agent Service** manages multi-step reasoning, tool execution, and prompt flows. State and chat history are securely offloaded to a dedicated **Azure Cosmos DB** instance.
4. **Secure Grounding (RAG):**
    - **Azure AI Search** stores chunked, vectorized enterprise documents. Data synchronization runs via private connectors, ensuring client documents are never exposed outside the corporate tenant.
5. **Evaluation & Metrics:**
    - F1 Score, BLEU, GLEU, METEOR, ROUGE
6. **Security & Networking:**
    - All backend traffic flows entirely over **Azure Private Endpoints**. No AI service or database exposes a public IP address. Egress traffic is strictly filtered via **Azure Firewall**. Microsoft security tools **Agent 365, Defender, Entra, Purview, Foundry, 365 Copilot, API management.**
7. **AI & Agentic Services:**
    - Foundry Agent Service
    - Foundry Model
    - Azure App services
    - Microsoft Agent framework
    - Microsoft 365
8. **Data**
    - Microsoft IQ
    - Microsoft Fabric
    - Azure Cosmos DB
    - Azure Databrics
    - Azure managed Redis
    - Azure Database for PostgreSQL
9. **Tools**
    - Azure Function
    - Azure Logic Apps

GCP Scalable AI Architecture (The Gemini Enterprise Platform Pattern)

This pattern leverages Google Cloud's unified AI ecosystem, utilizing the **Gemini Enterprise Agent Platform** (formerly Vertex AI), BigQuery for data warehousing, and Google Kubernetes Engine (GKE) for flexible compute hosting.

Plaintext



Key Components & Implementation Details

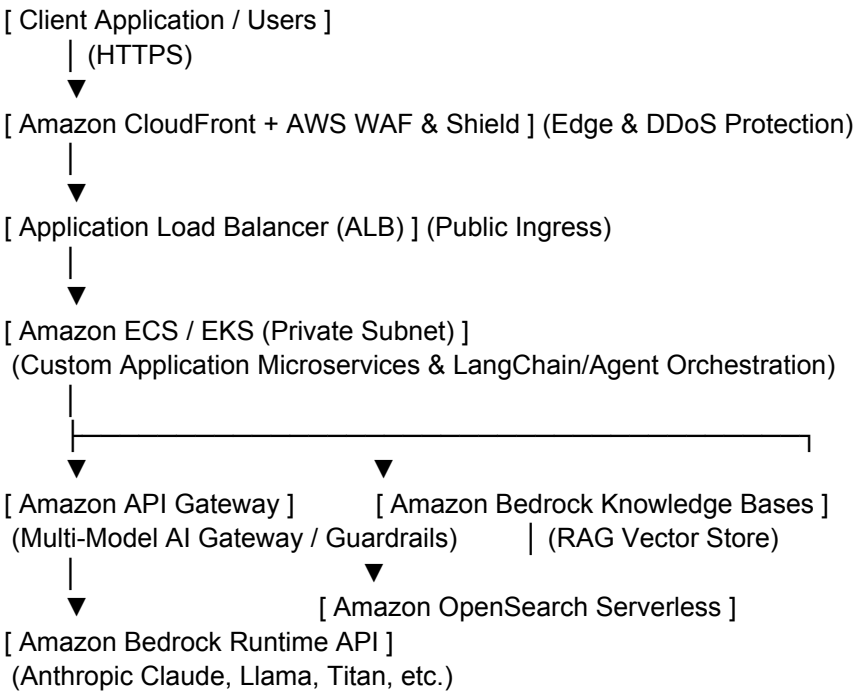
1. **Perimeter & Ingress:**
    - Requests pass through **Cloud Load Balancing** protected by **Cloud Armor** (GCP's enterprise DDoS and WAF service).
    - Microservices, custom backend logic, and frontend apps run on **Google Kubernetes Engine (GKE)**, providing elastic horizontal scaling for high-concurrency consulting workflows.
2. **Model Serving & Agent Ops:**
    - **Gemini Enterprise Agent Platform** acts as the central hub. Developers use **Model Garden** to access frontier models (like Gemini or third-party options like Anthropic Claude) and **Agent Studio** to construct agentic workflows.
    - Custom prediction routines and orchestration tools run natively inside GCP's managed pipelines.
3. **Enterprise Grounding & Data Synergy:**
    - **BigQuery** acts as both the analytical data warehouse and the vector database (using native vector search capabilities). This allows agents to execute real-time queries against structured financial data alongside unstructured documents stored in **Cloud Storage**.
4. **Security, Compliance & Guardrails:**
    - **Model Armor** provides native input/output filtering to strip PII, mitigate prompt injections, and block toxic outputs dynamically.
    - **VPC Service Controls (VPC-SC)** establish security perimeters around data projects, preventing data exfiltration even if an identity is compromised. Identity and Access Management (**IAM**) ensures granular role separation down to individual datasets and agent tools.

For **AWS**, a production-grade, secure, and scalable AI platform centers around **Amazon Bedrock** for model access, **Amazon ECS/EKS** for application compute, and an API Gateway pattern for enterprise governance.

AWS approaches this with a strong emphasis on network isolation via **AWS PrivateLink** and deep integration with native security primitives like **AWS IAM** and **AWS KMS**.

AWS Scalable AI Architecture (The Enterprise Bedrock Pattern)

Plaintext



Key Components & Implementation Details

1. **Perimeter & Ingress Security:**
    - Incoming user requests hit **Amazon CloudFront** protected by **AWS WAF** (Web Application Firewall) and **AWS Shield** to filter malicious payloads and mitigate DDoS attacks.
    - Traffic lands on an **Application Load Balancer (ALB)** sitting in a public subnet, which securely forwards traffic down to backend services running in private subnets.
2. **Compute & Orchestration Layer:**
    - Application business logic, user interfaces, and agent frameworks run on **Amazon ECS (Elastic Container Service)** or **Amazon EKS (Elastic Kubernetes Service)** deployed across private subnets for strict isolation.
3. **The AI Gateway & Governance:**
    - Rather than calling AI models directly, microservices route requests through an enterprise **AI Gateway pattern** (often built using Amazon API Gateway or open-source solutions like LiteLLM packaged for AWS). This handles global rate-limiting, tenant isolation, and unified cost tracking.
    - **Amazon Bedrock Guardrails** are applied inline to filter out harmful content, redact PII dynamically, and block hallucinations or prompt injections.
4. **Managed Model Access:**

- **Amazon Bedrock** provides serverless access to a wide choice of frontier models (Anthropic Claude, Meta Llama, Amazon Titan) through a single unified API.
- **Zero Data Retention Guarantee:** Client prompts and data sent to Bedrock are encrypted in transit and at rest (via **AWS KMS**) and are never used to train base foundation models.

5. **Secure Grounding (RAG):**
    - **Amazon Bedrock Knowledge Bases** handles the heavy lifting of document chunking, embedding generation, and synchronization.
    - Vectors are securely stored in **Amazon OpenSearch Serverless** or **Amazon Aurora PostgreSQL with pgvector**, protected by fine-grained access controls ensuring consultants only retrieve documents they are authorized to see.
6. **Network Isolation:**
    - Communication between your private VPC and Amazon Bedrock happens entirely through **AWS PrivateLink (Interface VPC Endpoints)**. Traffic never traverses the public internet, satisfying strict financial and professional services compliance frameworks.

A**WS vs. Azure vs. GCP: Enterprise Architecture Summary**

Dimension	**Microsoft Azure**	**Google Cloud (GCP)**	**Amazon Web Services (AWS)**
Primary AI Hub	Azure OpenAI & Microsoft Foundry	Gemini Enterprise Agent Platform	Amazon Bedrock
Vector Grounding	Azure AI Search	BigQuery Vector Search	Amazon OpenSearch Serverless
Network Security	Private Endpoints & Azure Firewall	VPC Service Controls & Cloud Armor	AWS PrivateLink & Security Groups
Core Advantage	Deep alignment with M365 and enterprise Entra ID workflows.	Massive analytical data synergy via BigQuery and TPU cost-efficiency.	Unmatched breadth of foundation model choices via a single unified API.